AUTOMATED LIBRARY MANAGEMENT SYSTEM

A Project Report

Submitted in partial fulfilment of the Requirements for the award of the Degree of

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

By

Khan Rukaiya Begum Asgar Ali & Gausiya Feroz Khan

16830 & 12786

Under the esteemed guidance of

PROF. CHANDRAKANT BORLE PROFESSOR

Designation



DEPARTMENT OF INFORMATION TECHNOLOGY

D.T.S.S. COLLEGE OF COMMERCE

(Affiliated to University of Mumbai)

Mumbai – 400 097

MAHARASHTRA

2025 – 2026

D.T.S.S. COLLEGE OF COMMERCE

(Affiliated to University of Mumbai)

MUMBAI - MAHARASHTRA - 400 097

DEPARTMENT OF INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project entitled, **AUTOMATED LIBRARY MANAGEMENT SYSTEM**, is Bonafide work of **(1) Khan Rukaiya Begum Asgar Ali, (2) Gausiya Feroz Khan** bearing Seat. No.: **16830 & 12786** submitted in partial fulfilment of the requirements for the award of degree of **Bachelor of Science** in **Information Technology** from University of Mumbai.

Internal Guide **Coordinator**

External Examiner

Date: **College Seal**

DECLARATION

I hereby declare that the project entitled, **"AUTOMATED LIBRARY MANAGEMENT SYSTEM"** done at **D.T.S.S. COLLEGE OF COMMERCE AND SCIENCE**, has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted this work to any other university.

The project is done in partial fulfilment of the requirements for the award of degree of **BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)** to be submitted as a final semester project as part of our curriculum.

Name: Khan Rukaiya Begum Asgar Ali & Gausiya Feroz Khan

Signature:_____&_____

ACKNOWLEDGEMENT

In completing this project report on the **"AUTOMATED LIBRARY MANAGEMENT SYSTEM"**, we had to take the help and guidelines of a few respected people, who deserve our greatest gratitude.

The completion of this project report gives us much pleasure. We would like to show our deepest gratitude to **MRS. HANEENA SONEY** for giving us excellent guidelines for the project throughout numerous consultations. We would also like to extend our sincere thanks to all those who have directly and indirectly guided us in writing this project report.

Many people, especially our classmates and friends, have made valuable comments and suggestions on this proposal which gave us the inspiration to improve our project. Here we thank all the people for their help, both directly and indirectly, in the successful completion of this project report.

(Khan Rukaiya Begum Asgar Ali), (Gausiya Feroz Khan)

ABSTRACT

This project showcases the development of a sophisticated web-based Automated Library Management System (ALMS) designed to solve the inefficiencies of traditional manual record-keeping. By replacing the labor-intensive "card and stamp" system with a digital request loop, the application enhances institutional productivity and provides students with seamless, real-time access to library resources.

Through rigorous planning, iterative architectural design, and meticulous coding, the application exemplifies proficiency in the Node.js runtime environment and the Bootstrap frontend framework. It demonstrates the ability to implement user-centered design principles by creating a mobile-first interface that caters to the modern student's needs.

Key features include real-time catalog updates, secure JWT-based authentication, and a cloud-integrated database via Neon (PostgreSQL). These features demonstrate technical skills in full-stack web development, relational data management, and the implementation of automated notification systems. The project also underscores a commitment to problem-solving, evidenced by overcoming challenges such as managing concurrent book reservation queues and ensuring data integrity across high-frequency transaction logs.

By presenting this project, I aim to showcase not only technical expertise in modern web technologies but also creativity, attention to detail, and a passion for building impactful digital solutions. This endeavor represents a significant milestone in my journey toward becoming a skilled full-stack developer prepared to contribute effectively to the field of automated resource management and software engineering.

Content

Chapter NO.	Title	Page No.
1.	INTRODUCTION	8
1.1	Background	8
1.2	Objectives	8
1.3	Purpose, Scope & Applicability	9
1.4	Achievement	9
2.	SURVEY OF TECHNOLOGY	10
2.1	Field Observation (The Manual System)	10
2.2	Statistical Analysis (Student Survey)	11
2.3	Conclusion of Survey	15
3.	REQUIREMENTS AND ANALYSIS	16
3.1	Problem Identification	16
3.2	Requirement Specification	16
3.3	Planning and Scheduling	17
3.4	Software and Hardware Requirements	20
3.5	Preliminary Product Description	22
3.6	Conceptual Models	23
4.	SYSTEM DESIGN	28
4.1	Basic Modules	28
4.2	Data Design (Database Schema)	29
4.3	Procedural Designs	33
4.4	User Interface Design	41
4.5	Security Issues	46
4.6	Test Cases Design	46
5.	IMPLEMENTATION AND TESTING	48
5.1	Implementation Approaches	48
5.2	Coding Details and Code Efficiency	49
5.3	Testing Approach	52
5.4	Modifications and Improvements	54
5.5	Test Cases	55
6.	RESULTS AND DISCUSSION	58
6.1	Test Reports	58

6.2	**User Documentation**	**60**
7.	**CONCLUSIONS**	**64**
7.1	**Conclusion**	**64**
7.2	**Limitations of the System**	**65**
7.3	**Future Scope of the Project**	**66**
8.	**REFERENCES**	**68**
9.	**GLOSSARY**	**70**

1. INTRODUCTION

In the modern academic landscape, digital transformation has redefined how educational resources are managed and accessed. The Automated Library Management System is a responsive web application designed to bridge the gap between traditional physical libraries and the need for instant, remote information access. Traditionally, library operations—such as checking book availability, maintaining member ledgers, and recording due dates—relied heavily on manual paperwork and physical presence. This manual "card and stamp" system is often time-consuming and prone to human error.

Our system introduces a two-way digital interface specifically optimized for mobile use, catering to both Students and Librarians. By leveraging a web-based portal, students can browse the complete library catalog, check real-time availability, and send digital checkout or reservation requests from anywhere. For the librarian, the system acts as a central administrative dashboard where they can approve requests, manage inventory, and monitor book circulation without the need for manual record books.

The core of this project is to eliminate unnecessary physical visits just for status inquiries. Through a robust backend and a mobile-friendly Bootstrap frontend, the application ensures that data is synchronized in real-time. Whether it is a student reserving a borrowed book or a librarian confirming a return, every transaction is logged digitally, ensuring accuracy and accountability in library management.

1.1 BACKGROUND

The traditional library system faces significant challenges in the digital age, primarily due to the "herculean task" of manual record maintenance. As educational institutions move toward paperless environments, there is a growing demand for systems that provide 24/7 accessibility. This project is engineered to replace the cumbersome manual process of flipping through record books with a streamlined, automated workflow. By digitizing library daily activities, we reduce the risks associated with lost files or damaged paperwork and significantly cut down the time spent on repetitive clerical tasks.

1.2 OBJECTIVES

- **Real-Time Accessibility:** To provide students with a 24/7 digital catalog to search for books and check availability from any mobile device.
- **Process Automation:** To automate the checkout and return cycle through a digital request-approval loop, minimizing manual data entry for librarians.
- **Operational Efficiency:** To eliminate the need for physical library cards and paper stamps by maintaining a secure, centralized digital database of all members and transactions.
- **Conflict Resolution:** To implement a reservation system that manages high-demand books fairly, allowing students to queue for books currently in use.

1.3 PURPOSE, SCOPE & APPLICABILITY

- **PURPOSE**

  The purpose of this web application is to provide a user-friendly, automated solution for college libraries. It aims to improve the productivity of both students and staff by providing a transparent system where book status and transaction history are always visible and accurate.

- **SCOPE**
  - **Catalog Management:** Digital storage of book details including title, author, and category. **Two-Way Request System:** Students can initiate checkout/return requests, and librarians can approve or reject them.
  - **Live Status Tracking:** Real-time visibility of whether a book is "Available," "Issued," or "Reserved".
  - **Reservation Queue:** Facility for users to reserve books that are currently checked out by others.
  - **Mobile-First Design:** Fully responsive interface using Bootstrap for seamless use on student and staff smartphones.

- **APPLICABILITY**

  This system is designed for college libraries and educational institutions that wish to modernize their resource management. It is particularly useful for students who need to manage their reading materials on the go and for librarians who require a centralized tool for efficient inventory and member supervision.

1.4 ACHIEVEMENT

The implementation of the Automated Library Management System achieves several key benefits for the institution:

- **Time & Resource Savings:** Eliminates the costs associated with printing and storing physical library forms and cards.
- **Data Integrity:** Centralizes student and book data in a secure database, preventing loss of records common in manual systems.
- **Enhanced Engagement:** Provides an intuitive search and recommendation interface that encourages students to utilize library resources more frequently.
- **Improved Decision Making:** Generates real-time reports and analytics on book usage, helping the library administration make informed purchasing decisions.

2. SURVEY OF TECHNOLOGY

The foundation of the Automated Library Management System is built upon a deep understanding of current limitations and user needs. To justify the shift from a traditional setup to a modern web-based ecosystem, a two-part research phase was conducted: a field observation of the existing manual system and a statistical survey involving 15 student participants.

2.1 FIELD OBSERVATION (THE MANUAL SYSTEM)

Direct field research was conducted at the college library to analyze the traditional "Card and Stamp" workflow. As shown in the captured field data, the current process is entirely paper-based and labor-intensive.

Traditionally, librarians are required to maintain manual records across two separate physical cards: one remains with the book (glued to the inner back cover), and another is kept in the library's administrative files. Every transaction involves handwritten entries of the "Date Due," "Borrower's Roll No," and "Return Date".

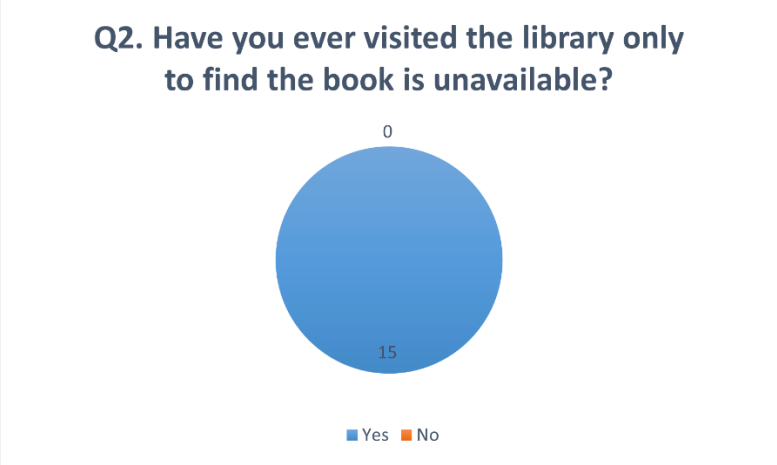
This manual record-keeping approach has several critical flaws:

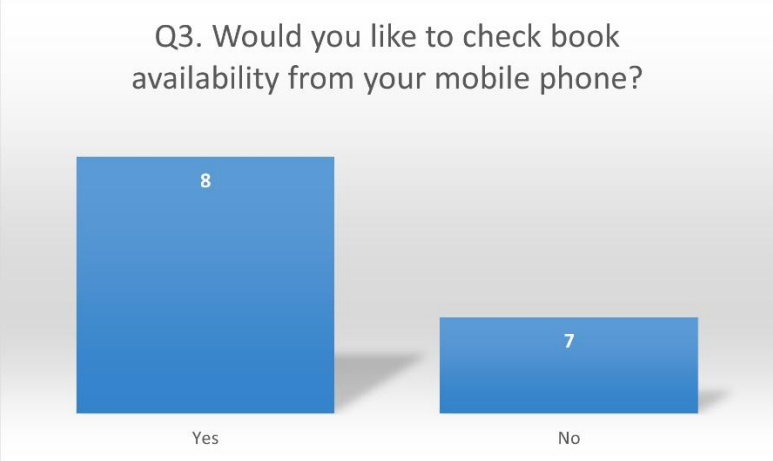
- **Inefficiency:** During peak hours, writing these details manually leads to long queues at the librarian's desk.
- **High Error Potential:** Illegible handwriting or manual entry mistakes can lead to inaccurate tracking of library assets.
- **Lack of Remote Access:** Students have no way of knowing if a book is available without physically visiting the library and checking the shelf.
- **Physical Degradation:** Constant handling of paper cards leads to wear and tear, risking the loss of historical transaction data.

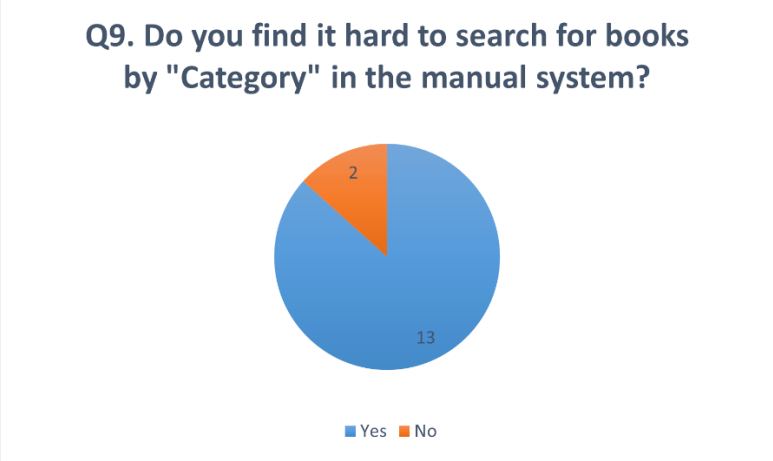
2.2 STATISTICAL ANALYSIS (STUDENT SURVEY)

To quantify the demand for a digital solution, a survey was conducted among 15 active library users. The data collected provides a clear mandate for the proposed system.

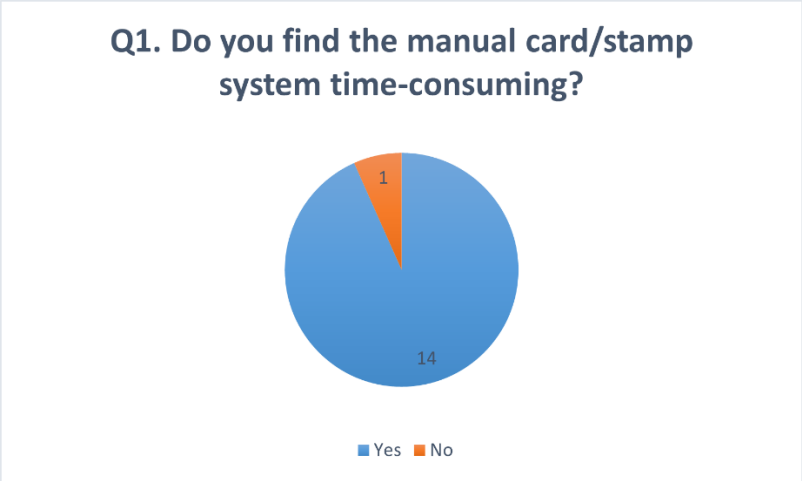
- **Availability and Search:** 100% of students reported that they have visited the library only to find that the book they needed was already issued. Furthermore, 86.7% find it difficult to search for books by category in the manual system.

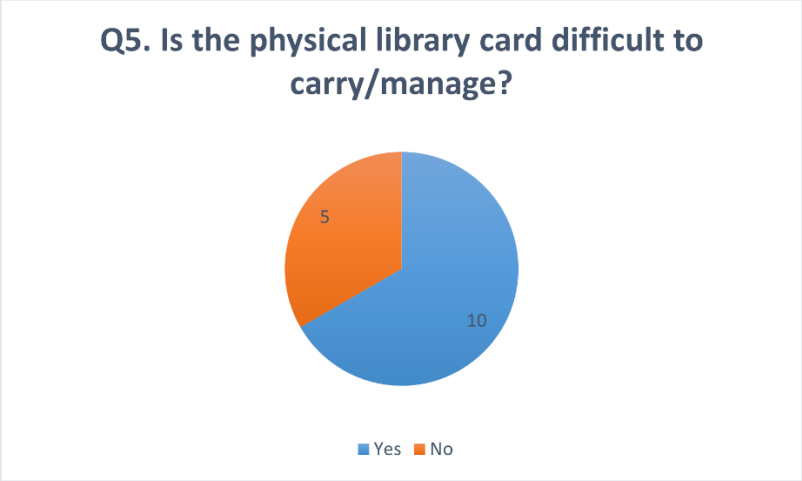


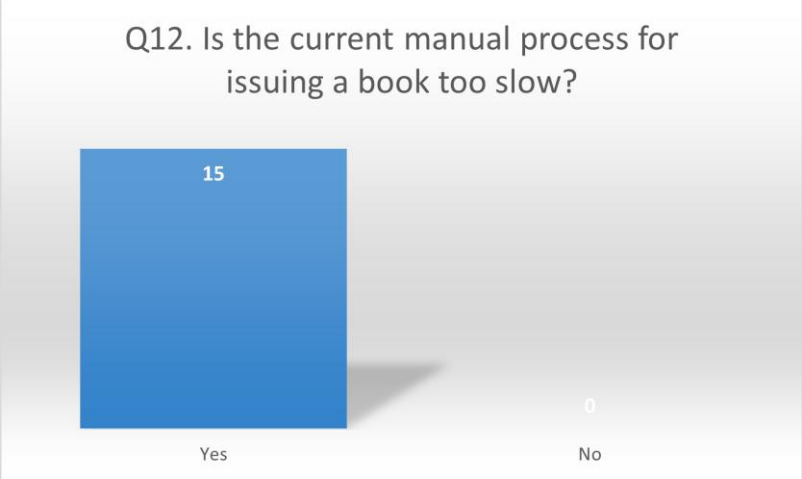




- **User Friction:** 93.3% of participants find the manual card/stamp system time-consuming and inefficient.

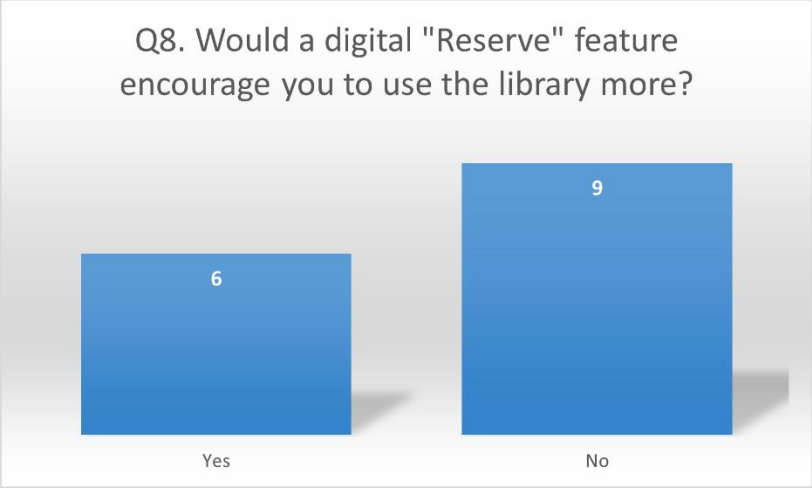


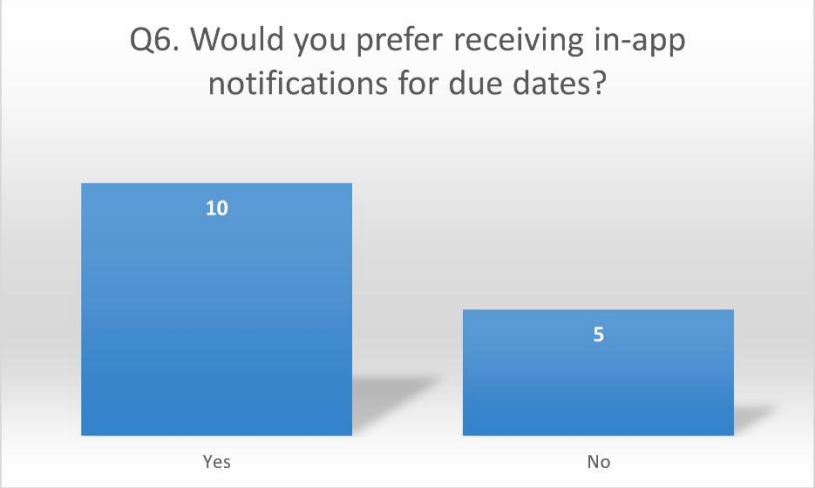


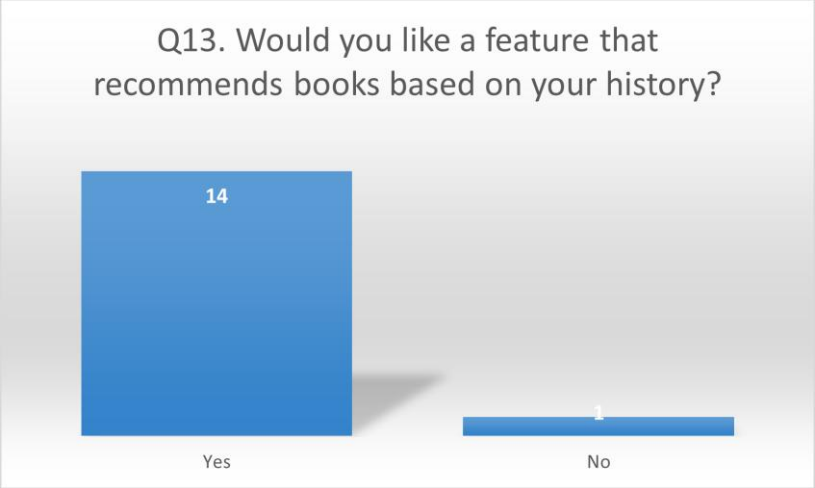


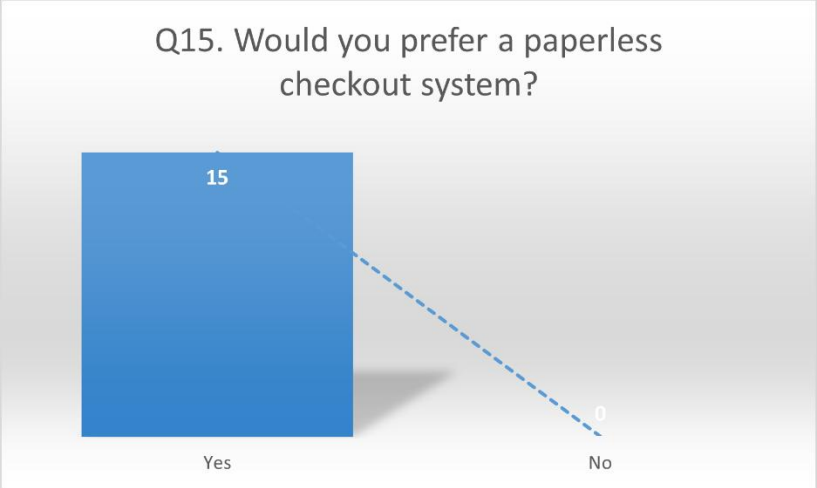


- **Digital Demand:** 100% of students expressed a desire for a mobile-responsive interface where they can check book availability and receive in-app notifications for due dates.

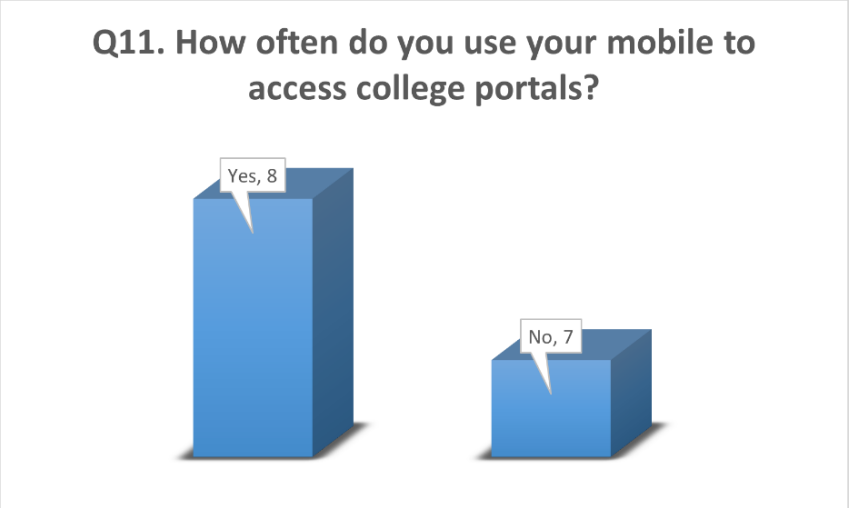


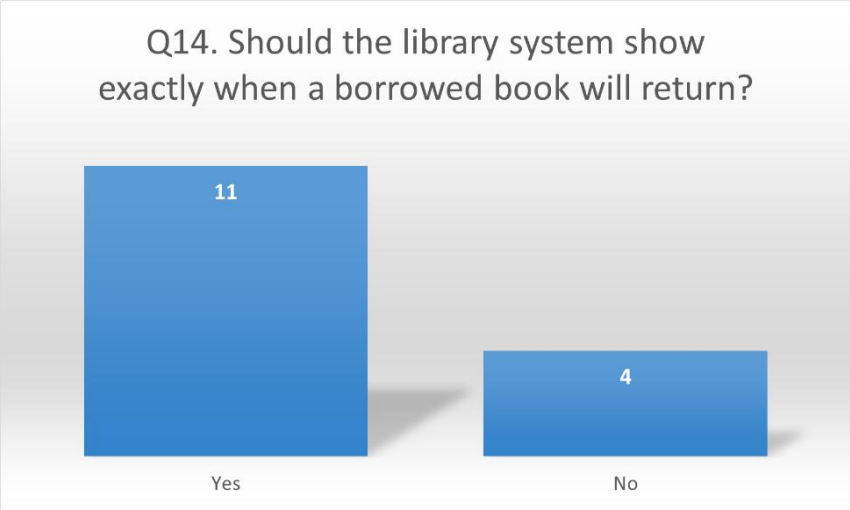



Q13. Would you like a feature that recommends books based on your history?
14
1
Yes
No


Q15. Would you prefer a paperless checkout system?
15
0
Yes
No

- **Mobile-First Readiness:** 100% of the surveyed group accesses the internet via mobile devices for more than 4-10 hours daily, confirming that a web app optimized for mobile is the ideal platform for this project.





2.3 CONCLUSION OF SURVEY

The results of the survey and field observations conclusively justify the development of the **Automated Library Management System**. With 93.3% of students seeking a digital alternative and 100% requiring remote availability checks, the current manual process is obsolete.

By utilizing the **Node.js, Bootstrap, and PostgreSQL (Neon)** stack, we can eliminate the physical friction of the "Card and Stamp" system. The automation of transaction history will not only reduce the librarian's clerical burden but also provide students with a modern, transparent, and high-speed resource management tool.

3. REQUIREMENT AND ANALYSIS

3.1 PROBLEM IDENTIFICATION

As the student population within the institution grows at a tremendous speed, the manual maintenance of library records has become increasingly difficult. The traditional "Card and Stamp" system is no longer sufficient to handle the volume of daily transactions. In the current manual system, a significant amount of time and manpower is required to record book issues, track due dates, and manage member ledgers.

The lack of real-time availability tracking often leads to wasted visits by students, as they have no way to verify if a book is on the shelf without being physically present. By moving to a computerized system, the need for manual paperwork is eliminated, and high accuracy is maintained in inventory tracking. Furthermore, maintaining a digital backup of library data is effortless and can be completed within a few minutes, ensuring that historical records are never lost due to physical damage.

3.2 REQUIREMENT SPECIFICATION

Functional Requirements

- **User Authentication:** The system provides secure and separate login portals for students and librarians to ensure data privacy.
- **Student Dashboard:** A dedicated interface where students can browse the digital catalog, send checkout/return requests, and view their current borrowed books.
- **Librarian Dashboard:** A central administrative panel to manage book inventory, approve or reject student checkout requests, and monitor return statuses.
- **In-App Notifications:** Instead of manual reminders, the system sends real-time digital alerts regarding request approvals, reservation status, and upcoming due dates.
- **Digital History Generation:** The system automatically generates transaction logs for every book, providing a clear history of who borrowed which book and when it was returned.

Non-Functional Requirements

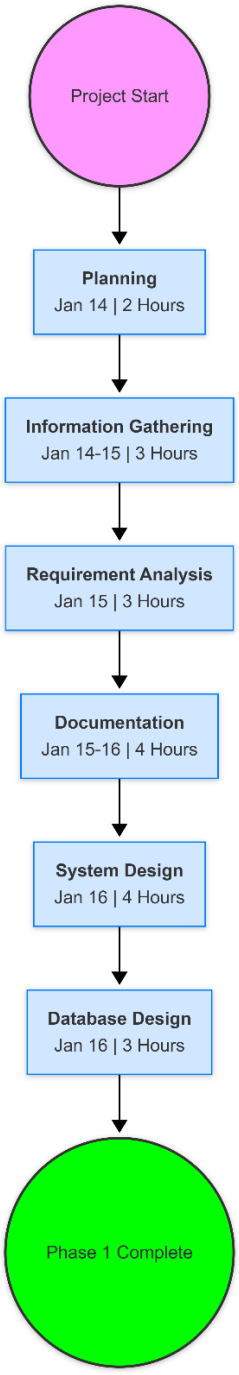
- **Scalability:** The architecture is designed to support an ever-growing number of library members and an expanding collection of books without performance degradation.
- **Security:** To protect user data, the system implements end-to-end encryption, role-based access control, and secure session management.
- **Performance:** The web application is optimized for low latency, ensuring that book searches and status updates happen in real-time.
- **Reliability:** By utilizing cloud-based hosting (Render/Neon), the system ensures 24/7 availability, allowing students to check the library catalog at any time from their mobile devices.

3.3 PLANNING AND SCHEDULING

To ensure the **Automated Library Management System** is built on a strong foundation, the project has been divided into distinct phases. For the current semester, we have completed the core research and blueprinting stages. This structured approach allows our two-student team to maintain accuracy and align our design with the institutional requirements.

Project Development Lifecycle (Phase 1)

The diagram below illustrates the sequential progression of our work. Each step was timed to ensure that the data gathered from the library visit was directly used to inform the system requirements and database schema.



Summary of Development Hours

TASK	START DATE	END DATE	DURATION
Planning	14 Jan	15 Jan	2 hours
Information Gathering	14 Jan	15 Jan	3 hours
Requirement Analysis	15 Jan	15 Jan	3 hours
Documentation	15 Jan	16 Jan	4 hours
Design	16 Jan	16 Jan	4 hours
Database Design	16 Jan	16 Jan	3 hours
Coding	Pending (Next Sem)	Pending (Next Sem)	Pending (Next Sem)
Integration	Pending (Next Sem)	Pending (Next Sem)	Pending (Next Sem)
Testing	Pending (Next Sem)	Pending (Next Sem)	Pending (Next Sem)
Deployment	Pending (Next Sem)	Pending (Next Sem)	Pending (Next Sem)

3.4 SOFTWARE AND HARDWARE REQUIREMENTS

To ensure the successful development and deployment of the **Automated Library Management System**, specific technical standards must be met. The following configurations outline the minimum requirements for a stable, high-performance web environment optimized for both desktop and mobile users.

SOFTWARE CONFIGURATION

Component	**Technology**
Frontend	HTML5, CSS3, JavaScript, and Bootstrap
Backend	Node.js (Express.js Framework)
Database	PostgreSQL (Hosted via Neon.tech)
Authentication	JWT (JSON Web Token) for Secure Sessions
Development IDE	VS Code with Agentic AI Extensions
Deployment	Render / Vercel (Cloud Hosting)

HARDWARE CONFIGURATION

Component	**Requirement**
Processor	Intel Core i3 / AMD Ryzen 3 or higher (Multi-core)
RAM	8 GB (Minimum for smooth multitasking)
Storage	256 GB SSD (Ensures fast read/write speeds for Node.js modules)
Internet	5 Mbps Stable Connection (For cloud database synchronization)

1. Development Hardware

- **Workstations:**
    - **CPU:** A multi-core processor is required to handle the concurrent running of the local server and AI-based development tools.
    - **RAM:** 8GB or more is highly recommended to manage modern web browsers, IDEs, and local background processes simultaneously.
    - **Storage:** An SSD is essential for faster project compilation and dependency installation (npm).
    - **Graphics:** Integrated graphics are sufficient as this is a text-based logic project without heavy 3D rendering requirements.

2. Networking Equipment

- **Internet Connection:** A reliable high-speed connection is necessary for downloading dependencies, pushing code to remote repositories (GitHub), and communicating with the Neon.tech cloud database.
- **Network Infrastructure:** Basic routing equipment is sufficient for managing local traffic during the initial testing phase between the student and librarian modules.

3. Testing Hardware

- **Mobile Devices:** Since the frontend utilizes Bootstrap for responsiveness, physical mobile devices or tablets are required to test the touch-interface and mobile-responsive layouts.
- **Testing Machines:** Standard workstations are used to verify cross-browser compatibility across Google Chrome, Mozilla Firefox, and Safari.
- **Cloud Server (Optional):** For final production testing, a cloud-based server environment (Render) is utilized to simulate real-world latency and remote data access scenarios.

3.5 PRELIMINARY PRODUCT DESCRIPTION

Product Name: Automated Library Management System (ALMS)

Overview: The **Automated Library Management System (ALMS)** is a comprehensive, two-way web-based platform designed to streamline the lifecycle of book circulation in educational institutions. By digitizing the traditional manual "card and stamp" process, ALMS offers a seamless and transparent experience for both students and librarians, ensuring high efficiency, data accuracy, and convenience.

An automated library system serves as a real-time digital bridge, managing both the inventory of the library and the transaction history of its members. Key capabilities of the system include:

- **Real-Time Catalog Search:** Students can browse the entire collection and check live availability without visiting the library premises.
- **Checkout & Return Requests:** A digital request-approval loop that replaces manual ledger entries and paper slips.
- **Availability Monitoring:** The system tracks the exact status of every book—whether it is on the shelf, issued to a user, or pending return.
- **Reservation Queue:** A built-in feature that allows students to reserve books currently in use, ensuring a fair and organized queue system.
- **In-App Notifications:** Automated digital alerts to notify users of request confirmations, reservation availability, and upcoming due date reminders.
- **Administrative Dashboard:** A centralized control panel for librarians to monitor all library activities, manage inventory, and generate usage reports at a glance.

The necessity for such a system is growing as educational institutions seek to optimize resources and reduce physical friction. ALMS ensures that the library's daily operations are conducted accurately and in a timely manner, significantly reducing the clerical burden on staff.

Intuitive User Interface:

- **Responsive Design:** A clean, mobile-first interface built with Bootstrap that ensures easy navigation on smartphones and tablets.
- **Step-by-Step Workflow:** Guided processes for book search, request initiation, and return confirmation with clear progress indicators.

Inventory & Member Management:

- **Dynamic Catalog:** Librarians can easily add new books, update categories, and manage author details through the administrative portal.
- **Secure Member Records:** Organized storage of student data, including their current borrowed books and complete transaction history.

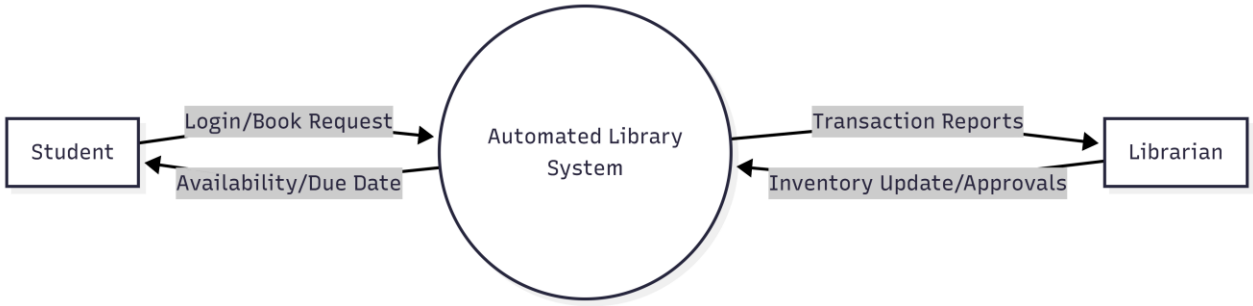
Real-Time Transaction Tracking:

- **Live Status Updates:** Students can track their borrowed books in real-time, viewing return deadlines and current request statuses.
- **Operational Visibility:** Librarians can monitor the circulation of books and review pending requests through an intuitive, data-driven dashboard.

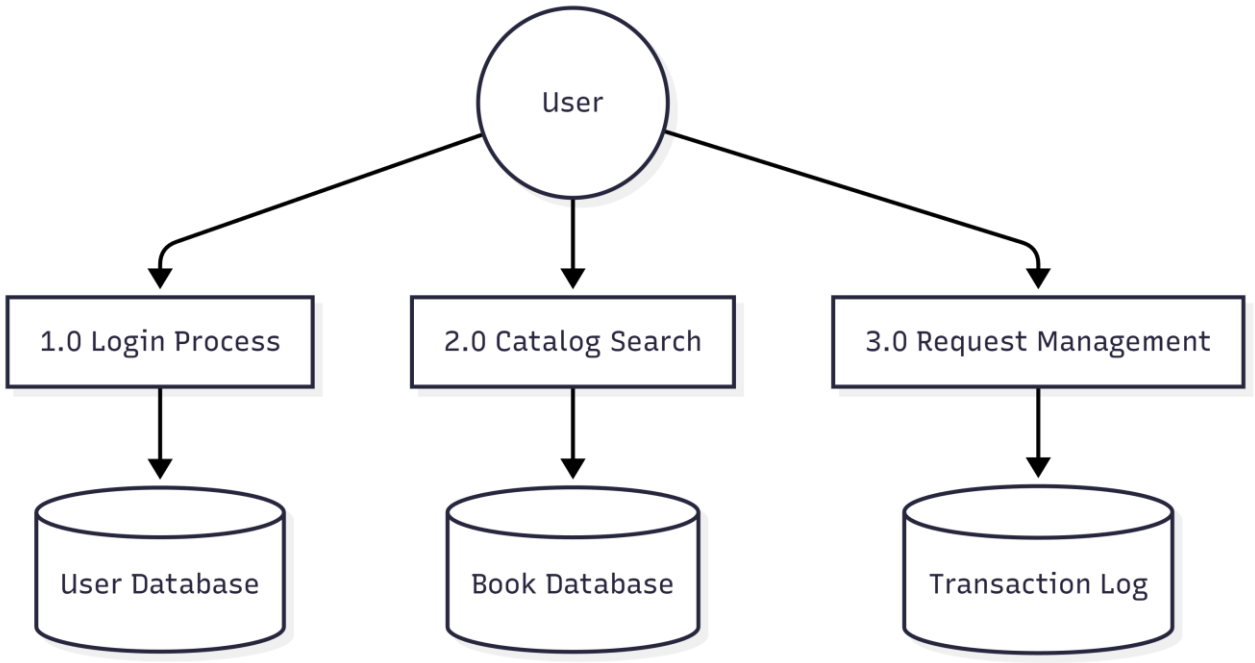
3.6 CONCEPTUAL MODELS

3.6.1 DATA FLOW DIAGRAM (DFD)

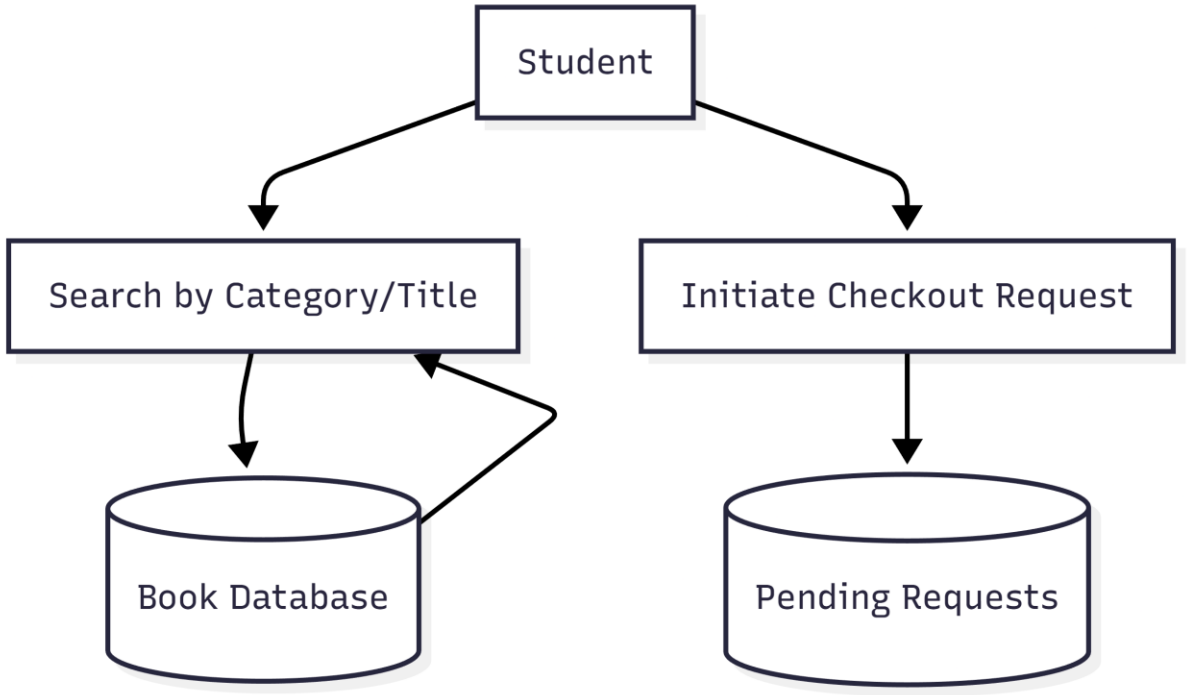
Level 0 (Context Level DFD) The Level 0 diagram represents the entire library system as a single process, showing the external entities (Student and Librarian) and their basic data exchanges with the system.



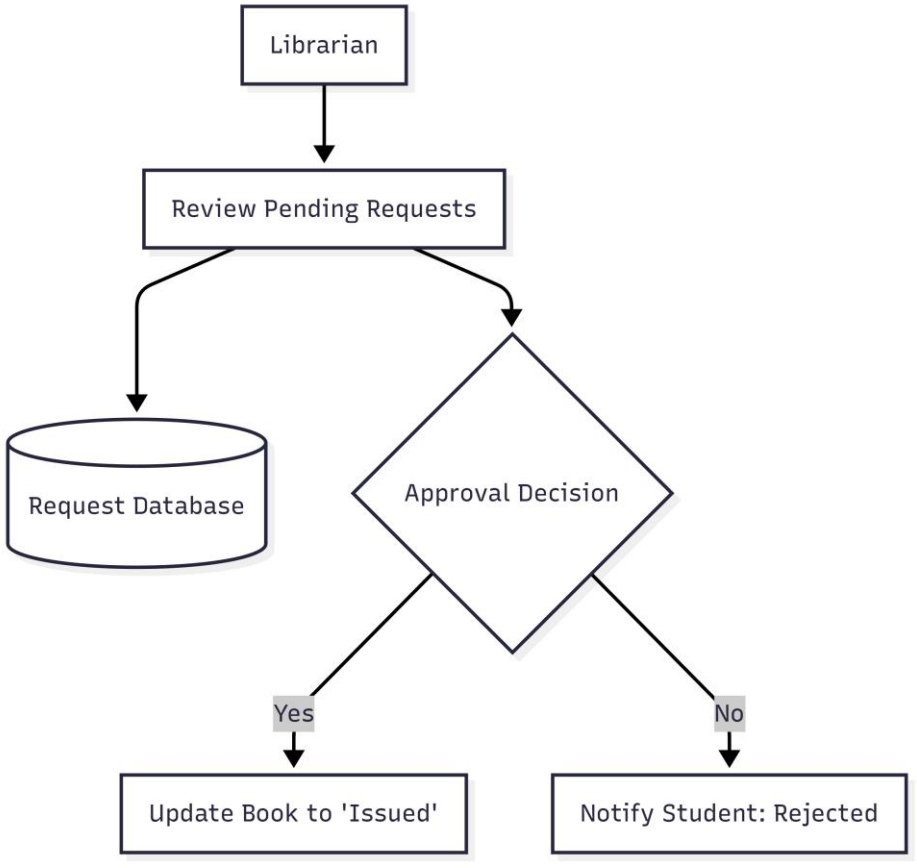
Level 1 (Functional DFD) Level 1 breaks the system into its core functions: Login, Catalog Management, and Request Processing.



DFD Level 2 (Student Interaction Flow) This level zooms into the Student's specific actions, such as searching for a book and sending a checkout request.

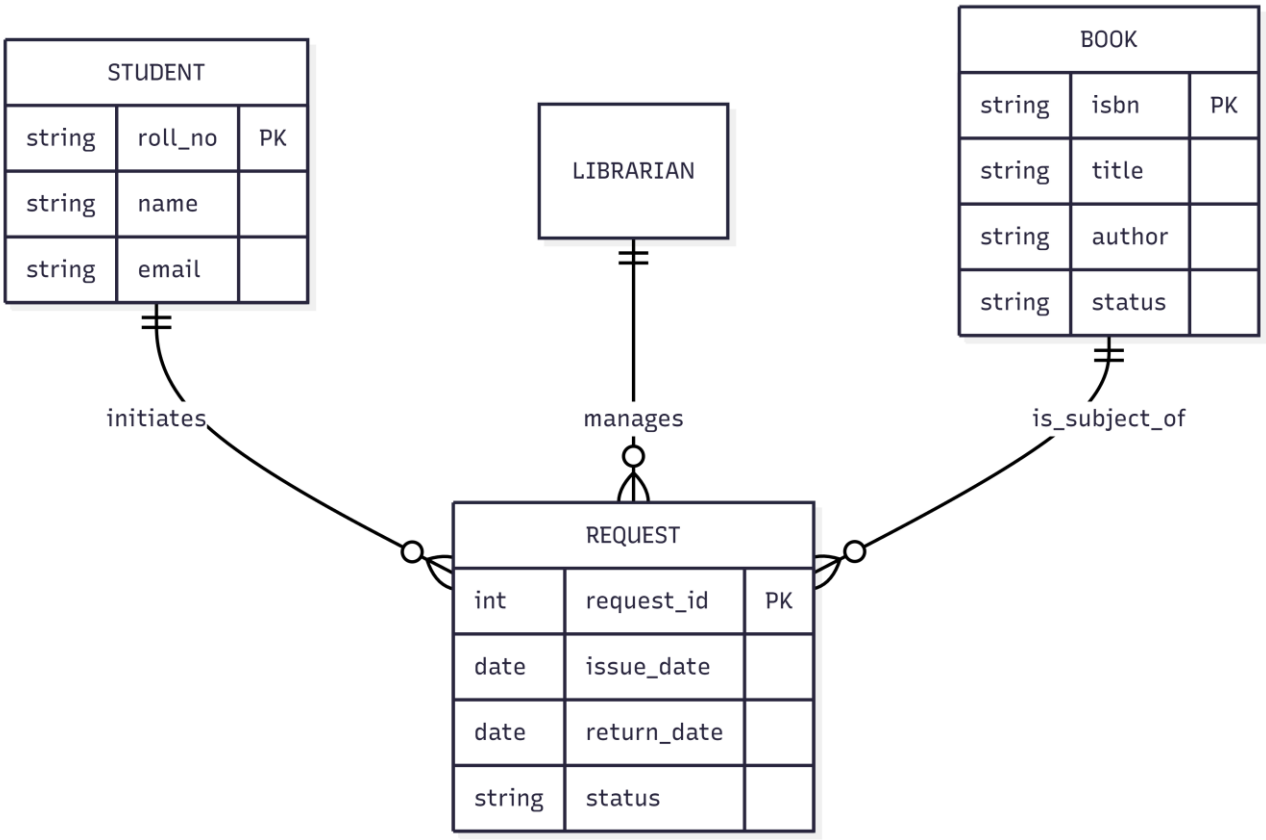


DFD Level 3 (Librarian Admin Flow) Level 3 focuses on the Librarian's back-end management, specifically how they approve requests and update book status.



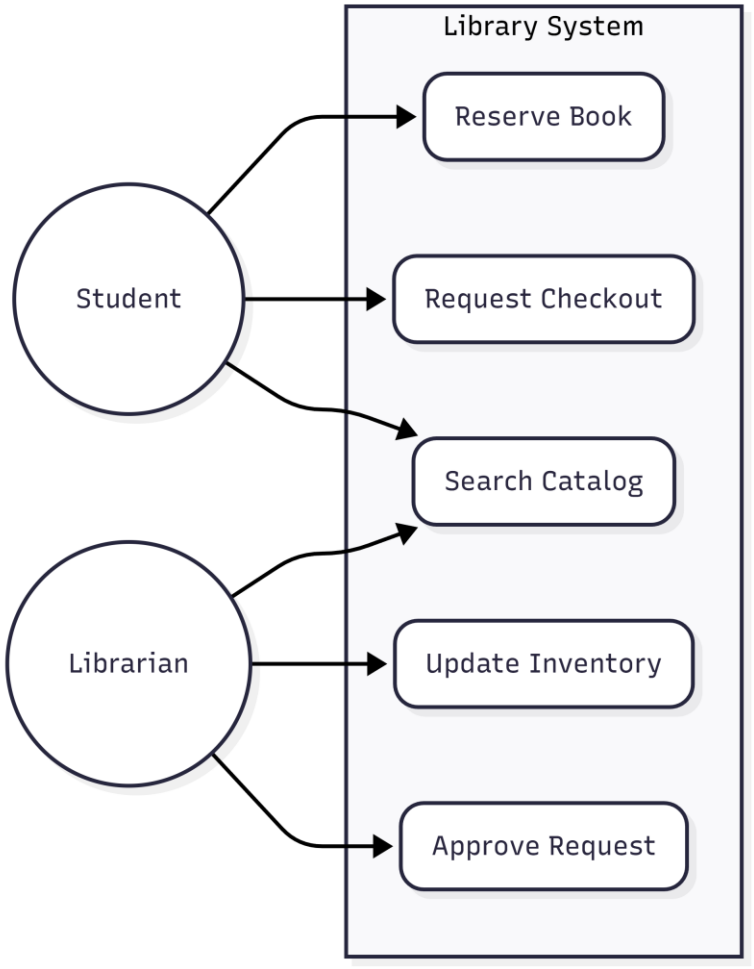
3.6.2 ER DIAGRAM (Entity Relationship Diagram)

The ER Diagram defines the logical structure of the database. It illustrates the relationships between core entities like Students, Books, and the Librarian.



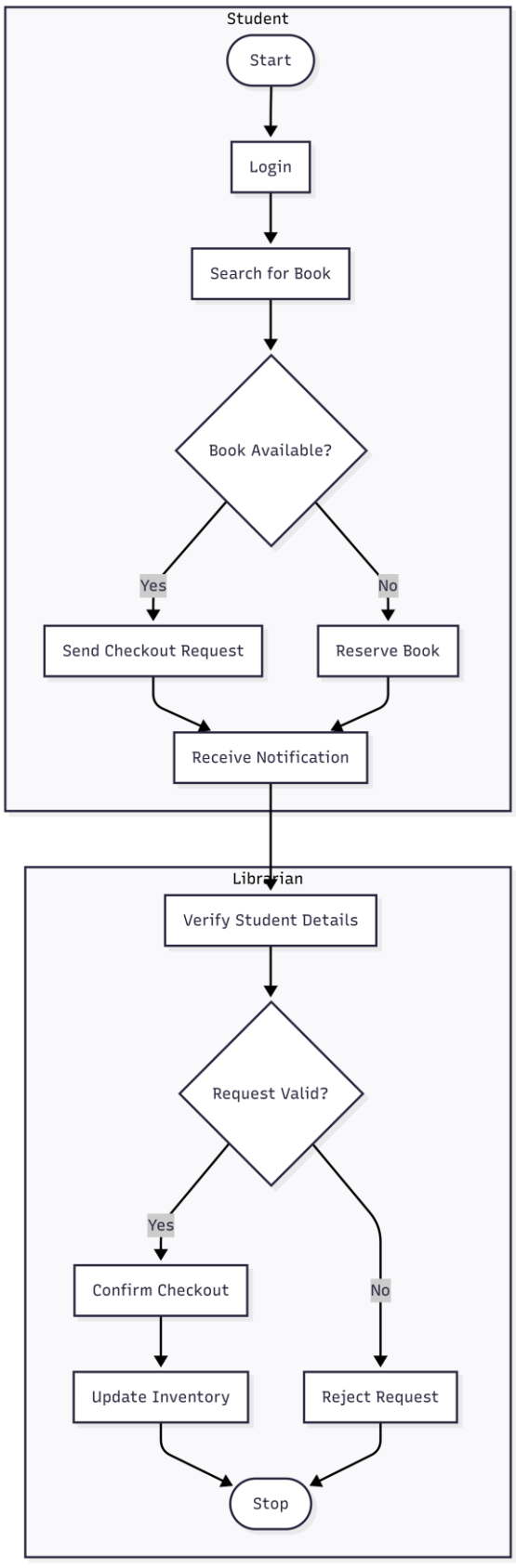
3.6.3 USE CASE DIAGRAM

This diagram defines the scope of the project by listing exactly what each user can do within the application.



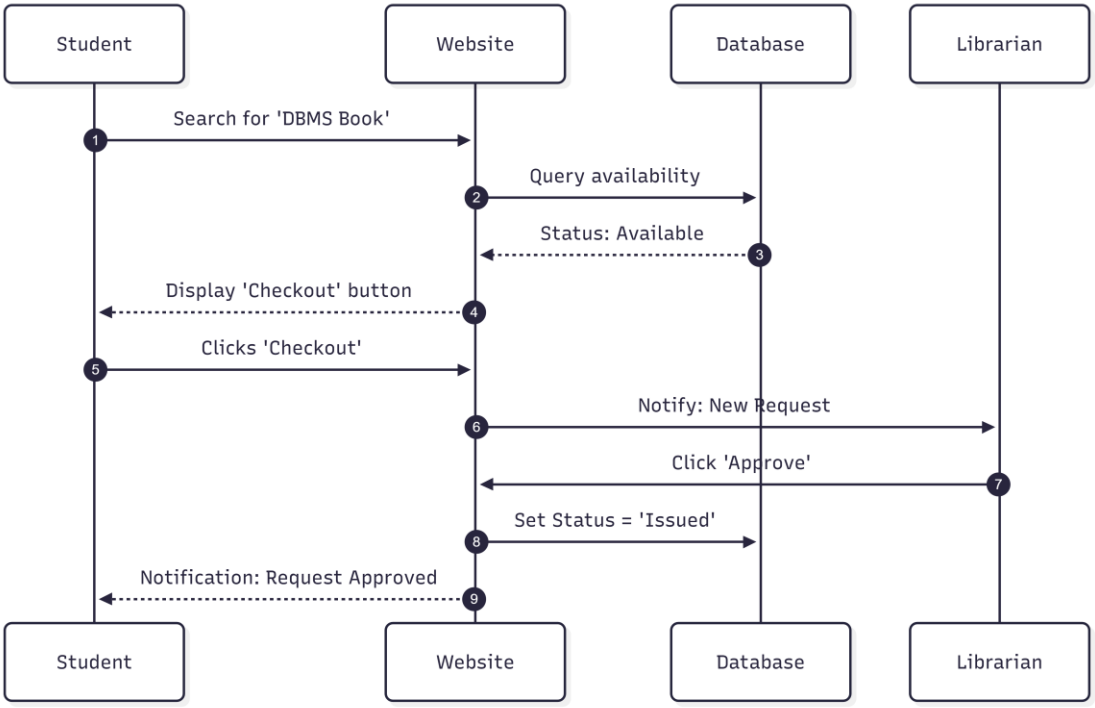
3.6.4 ACTIVITY DIAGRAM

An activity diagram is essential because it shows the **logic flow**—the decisions made by the system (e.g., "Is the book available?").



3.6.5 SEQUENCE DIAGRAM

This diagram shows the **time-ordered** interaction between the User, the Website, and the Database for a single transaction.



4. SYSTEM DESIGN

The system design phase transforms the conceptual requirements into a technical blueprint. This software architecture has been fine-tuned to ensure high performance, real-time data accuracy, and a mobile-responsive user experience. The design determines how input data (book requests) is processed to produce the desired output (status updates and transaction logs).

4.1 BASIC MODULES

Module 1: User Authentication

- **Registration & Login:** Secure portals for Students and Librarians using email and password.
- **Role-Based Access:** System automatically redirects users to either the Student Dashboard or Librarian Panel based on their credentials.

Module 2: Digital Catalog

- **Book Search:** Students search by Title, Author, or Category.
- **Live Status:** Real-time display of whether a book is "Available," "Issued," or "Reserved."

Module 3: Request Management

- **Checkout Initiation:** Students send a digital request to borrow a book.
- **Reservation System:** If a book is issued, students can join a digital queue.
- **Approval Loop:** Librarians receive alerts to approve or reject pending requests.

Module 4: In-App Notifications

- **Status Alerts:** Real-time updates when a librarian approves a checkout.
- **Due Date Reminders:** Automated dashboard alerts for books nearing their return deadline.

Module 5: Administrative Panel

- **Inventory Control:** Librarians can add, edit, or remove books from the system.
- **Transaction Tracking:** A complete view of all active issues and historical returns.

4.2 DATA DESIGN (Database Schema)

1: USERS (Student & Librarian Data)

Field Name	Data Type	Description
user_id	SERIAL (PK)	Unique ID for every user
name	VARCHAR(255)	Full name of the user
email	VARCHAR(255)	Unique login email
password	VARCHAR(255)	Hashed password
role	VARCHAR(50)	'student' or 'librarian'
mob_number	VARCHAR(15)	Mobile number for contact
created_at	TIMESTAMP	Time of registration

2: BOOKS (Inventory)

Field Name	Data Type	Description
book_id	SERIAL (PK)	Unique book ID
title	VARCHAR(255)	Book title
author	VARCHAR(255)	Author name
category	VARCHAR(100)	e.g., Fiction, Science, History
status	VARCHAR(50)	Available, Issued, Reserved
shelf_loc	VARCHAR(50)	Physical location in library

3: REQUESTS (Checkout / Return Loop)

Field Name	Data Type	Description
request_id	SERIAL (PK)	Unique request ID
student_id	INT (FK)	References USERS(user_id)
book_id	INT (FK)	References BOOKS(book_id)

Field Name	Data Type	Description
req_type	VARCHAR(50)	‘checkout’ or ‘return’
status	VARCHAR(50)	Pending, Approved, Rejected
created_at	TIMESTAMP	Time request was made

4: TRANSACTIONS (Active Issues)

Field Name	Data Type	Description
trans_id	SERIAL (PK)	Unique transaction ID
student_id	INT (FK)	Who has the book
book_id	INT (FK)	Which book is issued
issue_date	DATE	Date of approval
due_date	DATE	Expected return date
status	VARCHAR(50)	Issued, Returned, Overdue

5: RESERVATIONS (Queue Management)

Field Name	Data Type	Description
res_id	SERIAL (PK)	Unique reservation ID
student_id	INT (FK)	Student in queue
book_id	INT (FK)	Book being reserved
queue_pos	INT	Position in the waitlist
expiry_date	TIMESTAMP	Date when reservation expires

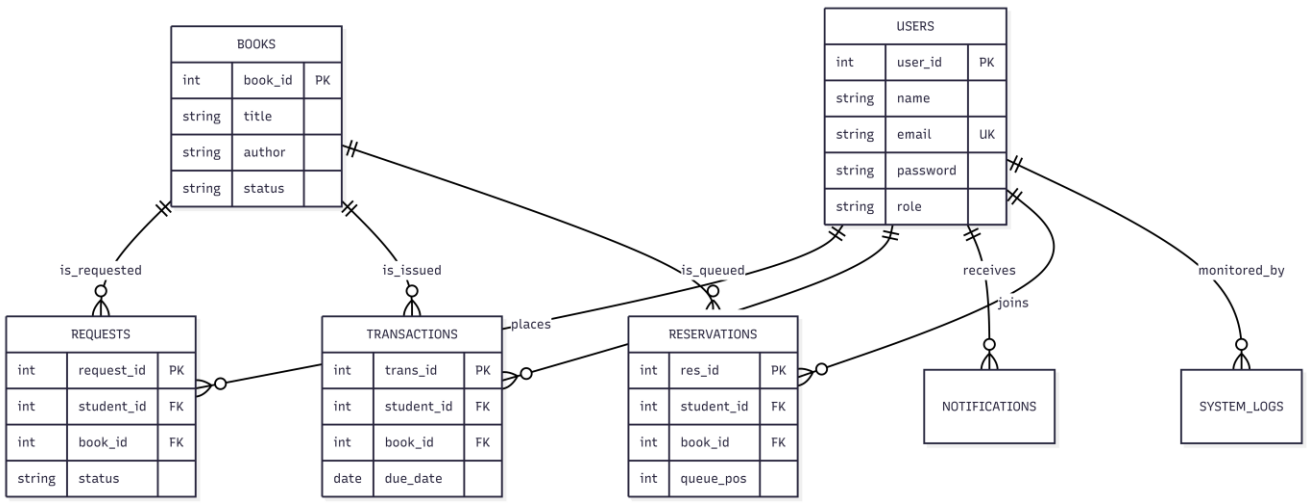
6: NOTIFICATIONS

Field Name	Data Type	Description
note_id	SERIAL (PK)	Unique notification ID
user_id	INT (FK)	Target recipient
message	TEXT	Content of the alert
is_read	BOOLEAN	Read / Unread status
created_at	TIMESTAMP	Time alert was sent

7: SYSTEM_LOGS (Analytics)

Field Name	Data Type	Description
log_id	SERIAL (PK)	Unique log ID
action	VARCHAR(255)	e.g., ‘Inventory Updated’, ‘Book Returned’
admin_id	INT (FK)	Librarian who performed action
timestamp	TIMESTAMP	Time of action

4.2.1 SCHEMA DESIGN



4.2.2 DATA INTEGRITY AND CONSTRAINTS

Data integrity and constraints are crucial for maintaining accuracy, consistency, and reliability in the **Automated Library Management System**. These rules ensure that book statuses, student records, and transaction logs remain synchronized and error-free.

- **Data Integrity:**
    1. **Entity Integrity:** Ensures each entity (e.g., Users, Books) has a unique identifier (Primary Key) to prevent duplicate records.
    2. **Referential Integrity:** Ensures relationships between entities are consistent; for example, a checkout request must reference a valid student ID.
    3. **Domain Integrity:** Ensures data values fall within defined ranges, such as restricting the "Status" field to predefined terms.
    4. **User-Defined Integrity:** Implements custom rules, such as preventing a student from requesting more than three books at once.
- **Constraints:**
    1. **Primary Key (PK) Constraint:** Unique identifier for every row in every table.
    2. **Foreign Key (FK) Constraint:** Maintains the link between tables, like linking a transaction to a book ID.
    3. **Unique Constraint:** Ensures values like student email addresses are not duplicated.
    4. **Not Null Constraint:** Ensures essential fields, such as "Book Title," are never left empty.
    5. **Check Constraint:** Validates that return dates are always later than issue dates.
    6. **Default Constraint:** Automatically sets a new book's status to 'Available' upon entry.
- **Automated Library System Constraints:**
    1. **Users Table:** PK: user_id; Unique: email; Not Null: name, email, role.
    2. **Books Table:** PK: book_id; Not Null: title, author, category.
    3. **Requests Table:** PK: request_id; FK: user_id, book_id; Not Null: req_type, status.
    4. **Transactions Table:** PK: trans_id; FK: user_id, book_id; Not Null: issue_date, due_date.
    5. **Reservations Table:** PK: res_id; FK: user_id, book_id; Not Null: queue_pos.

- **Data Validation:**
    1. **Format Validation:** Ensures email formats and phone numbers match standard patterns.
    2. **Range Validation:** Ensures the number of days for borrowing falls within the institutional limit.
    3. **Length Validation:** Restricts password and username lengths for security purposes.
- **Data Normalization:** Normalization is used to separate redundant fields and break large tables into smaller, efficient ones to avoid update anomalies. All tables are normalized to the **Third Normal Form (3NF)**:
    - **1NF:** Ensures all attributes contain simple individual values.
    - **2NF:** Eliminates partial dependencies; all non-key attributes depend on the primary key.
    - **3NF:** Eliminates transitive dependencies to ensure high data integrity.

4.3 PROCEDURAL DESIGNS

- **User Registration Procedure:**
    1. User submits registration form (Name, Email, Roll No, Password).
    2. System checks for existing email or Roll No in the PostgreSQL database.
    3. System hashes the password for security and creates a new user profile.
- **Book Request Procedure:**
    1. Student searches for a book and clicks "Request Checkout".
    2. System verifies if the student has no overdue books or unpaid fines.
    3. A pending request is generated and displayed on the Librarian’s dashboard.
- **Librarian Approval Procedure:**
    1. Librarian reviews the digital queue of "Pending Requests".
    2. Upon physical handover of the book, Librarian clicks "Approve".
    3. System updates book status to 'Issued' and generates a due date in the Transactions table.
- **Book Return Procedure:**
    1. Student selects an active issue from their dashboard and clicks "Initiate Return".
    2. Librarian verifies the book's condition and confirms the return.

3. System marks the book as 'Available' and notifies any students in the reservation queue.

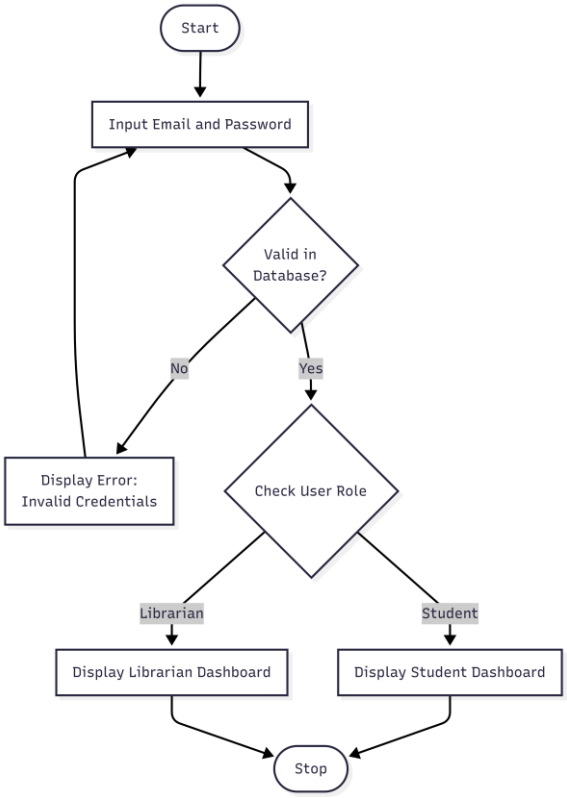
- **Communication Procedure:**
    1. System sends automated in-app notifications for:
        - Checkout request approvals.
        - Due date reminders (24 hours before deadline).
        - Reservation availability alerts.
- **Security and Authentication Procedure:**
    1. System uses SSL/TLS encryption for all data transmission.
    2. Implementation of **JWT (JSON Web Tokens)** for secure, session-based login.
    3. Strict role-based access control (RBAC) to separate Student and Librarian functions.
- **Error Handling and Backup Procedure:**
    1. System logs all exceptions and displays user-friendly error messages.
    2. Automated daily backups are performed via the Neon.tech cloud database.
    3. Redundancy ensures data can be restored instantly in case of loss.

4.3.1 LOGIC DIAGRAMS

Control Flow Chart The logic diagrams below represent the decision-making process within the application. These charts ensure that user inputs are validated before any changes are committed to the PostgreSQL database.

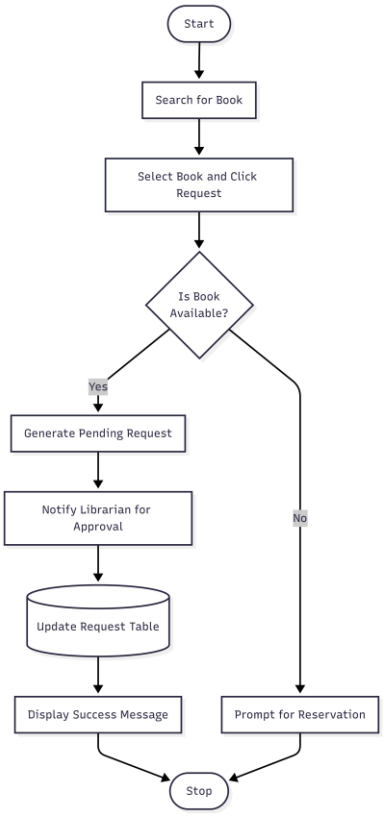
1. Login Form Logic

This flowchart illustrates the authentication process for both Students and Librarians. It checks the credentials against the **Users** table and redirects the user to their specific dashboard based on their role.



2. Book Request (Checkout) Form Logic

This diagram shows the procedural logic when a student requests a book. It includes a validation step to ensure the book is currently available before allowing a request to be generated in the **Requests** table.



4.3.2 DATA STRUCTURES

Table Structure In Details: The following tables define the physical data storage structure for the PostgreSQL database. Each table is designed to maintain relational integrity and support the two-way request system

1: USERS_MST (User Master)

Field Name	Data Type	Description	Constraints
user_id	INT	Unique User ID	Primary Key
name	VARCHAR(255)	Full Name	Not Null
email	VARCHAR(255)	Login Email	Unique, Not Null
password	VARCHAR(255)	Hashed Password	Not Null
role	VARCHAR(50)	'student' / 'librarian'	Not Null

2: BOOKS_MST (Inventory Master)

Field Name	Data Type	Description	Constraints
book_id	INT	Unique Book ID	Primary Key
title	VARCHAR(255)	Book Title	Not Null
author	VARCHAR(255)	Author Name	Not Null
category	VARCHAR(100)	Book Genre	Not Null
status	VARCHAR(50)	'Available' / 'Issued'	Default 'Available'

3: REQUEST_MST (Transaction Requests)

Field Name	Data Type	Description	Constraints
req_id	INT	Request ID	Primary Key
user_id	INT	Student ID	Foreign Key
book_id	INT	Book ID	Foreign Key
req_type	VARCHAR(50)	'Checkout' / 'Return'	Not Null
status	VARCHAR(50)	'Pending' / 'Approved'	Not Null

4. Technical Architecture Overview

1. **Relational Database Management System (RDBMS):**
    - **User Table:** Stores core profile data (ID, name, role).
    - **Books Table:** Stores catalog information and live availability status.
    - **Transaction Table:** Stores approved checkouts and return deadlines.
2. **Object-Oriented Classes:**
    - **Student Class:** Methods: searchBook(), requestCheckout(), viewNotifications().
    - **Librarian Class:** Methods: approveRequest(), updateInventory(), manageUsers().
3. **Key Considerations:**
    - **Security:** Password hashing and JWT-based session management.
    - **Scalability:** Hosted on Neon (PostgreSQL) for high-performance cloud data handling.
    - **Integrity:** Strict Foreign Key constraints to prevent orphaned transaction records.

4.3.3 ALGORITHMS DESIGN

Algorithm: Library Book Circulation Processing

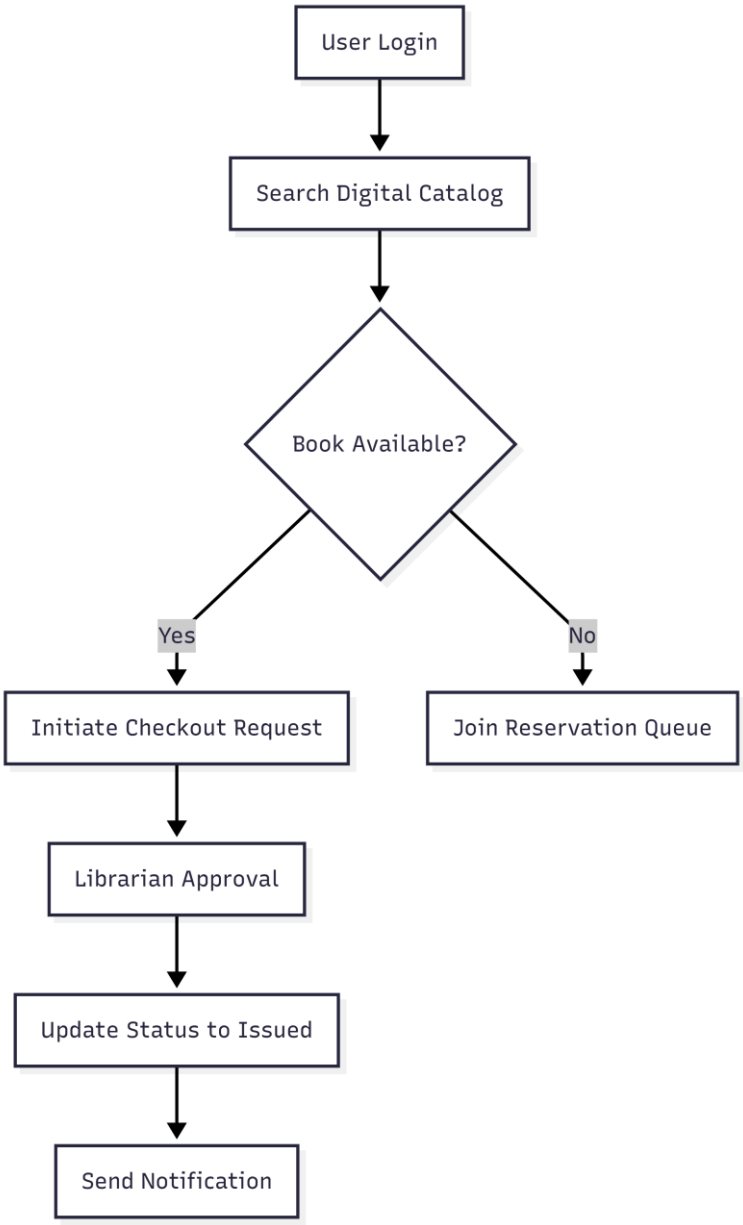
- **Input Data:**
    1. **User Credentials:** Email and Password for authentication.
    2. **Search Queries:** Title or Category for catalog filtering.
    3. **Request Details:** Book ID and Request Type (Checkout/Return).
- **Output Data:**
    1. **Request Status:** Success message or "Book Already Issued" error.
    2. **Notification:** Approved checkout alert with due date.
    3. **Updated Catalog:** Real-time change in book availability.

Procedural Process:

- **Step 1: Authentication:** Validate user credentials and identify role (Student/Admin).
- **Step 2: Catalog Search:** Filter database for books matching user input.
- **Step 3: Request Initiation:** If status is 'Available', generate a 'Pending' entry in Request Table.

- **Step 4: Librarian Approval:** Verify student eligibility and physically hand over book; update status to 'Issued'.
- **Step 5: Return Loop:** Student initiates return; Librarian confirms receipt; system resets book status to 'Available'.

Algorithm Flowchart:



Pseudocode:

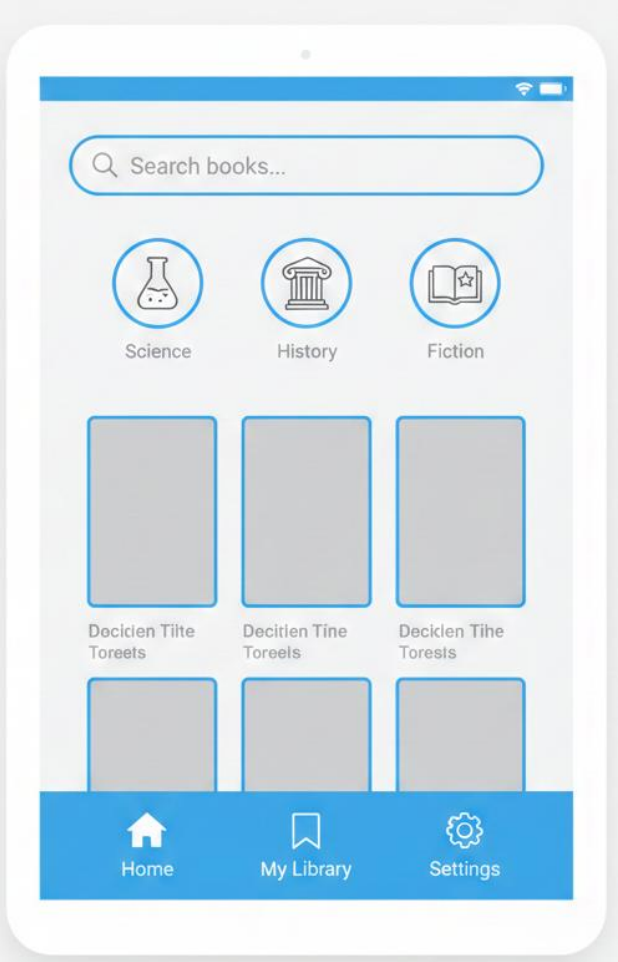
```
def library_circulation_processing():
    user_authentication()
    catalog_search()
    checkout_request()
    librarian_approval()
    inventory_update()
    send_notification()


def checkout_request():
    # Check if user has overdue books
    # If eligible, create pending request in DB
    # Update UI to 'Request Sent'


def librarian_approval():
    # Review pending requests list
    # On physical handover, click 'Approve'
    # Change book status in BOOKS_MST
```

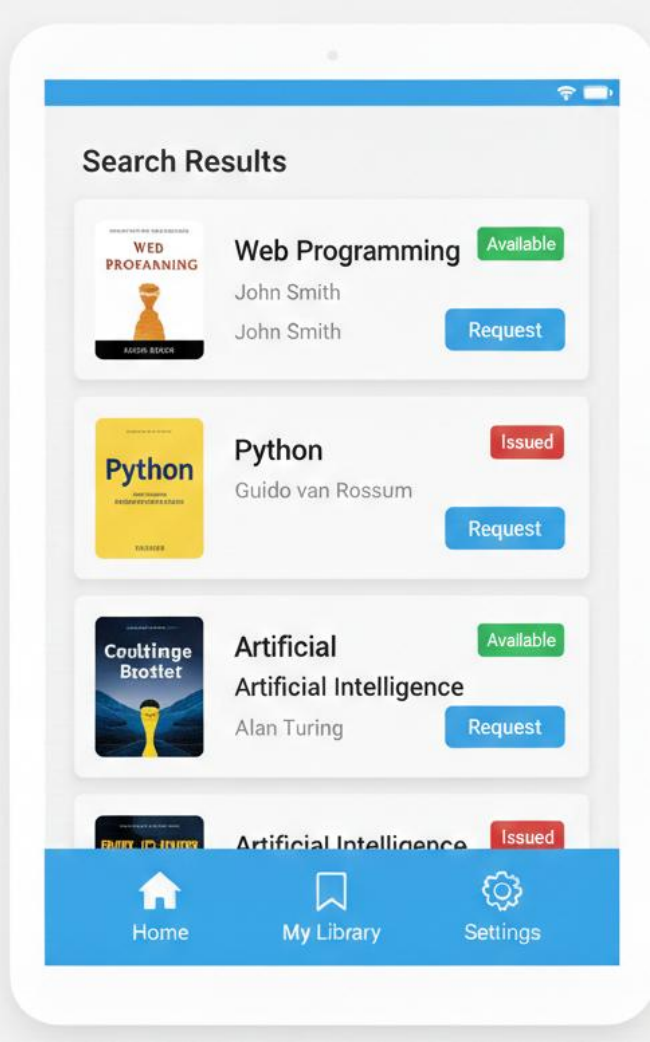
4.4 USER INTERFACE DESIGN

The user interface of the **Automated Library Management System** is designed with a "Mobile-First" approach using the **Bootstrap** framework. The goal is to provide a frictionless experience that replaces the slow manual card system with an intuitive digital request loop.



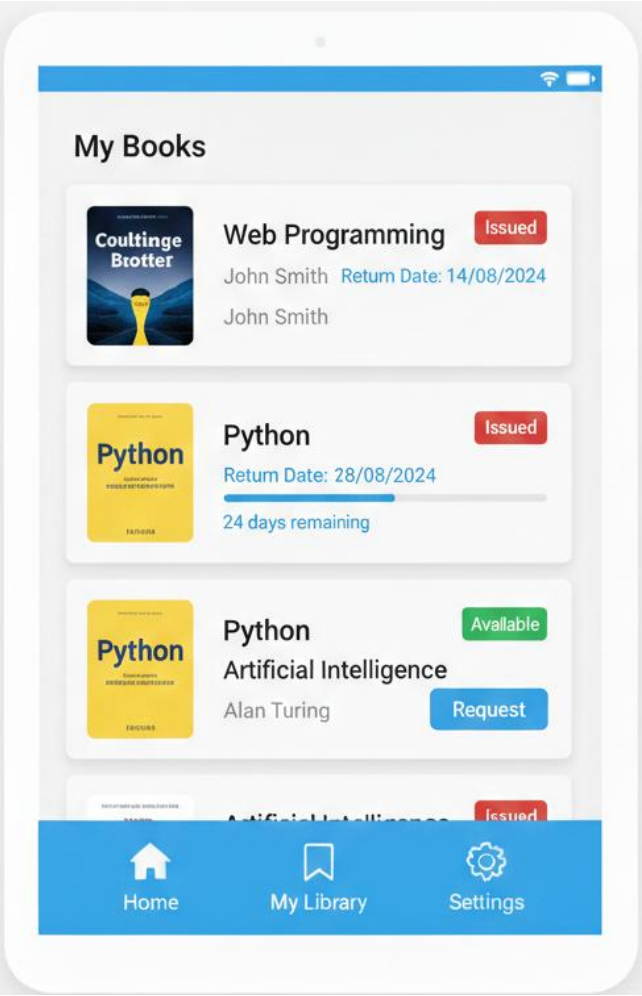
4.4.1 Student Portal Design

The student interface prioritizes ease of access and transparency. It allows users to manage their entire library lifecycle—from searching for a book to tracking return deadlines—without physical intervention.



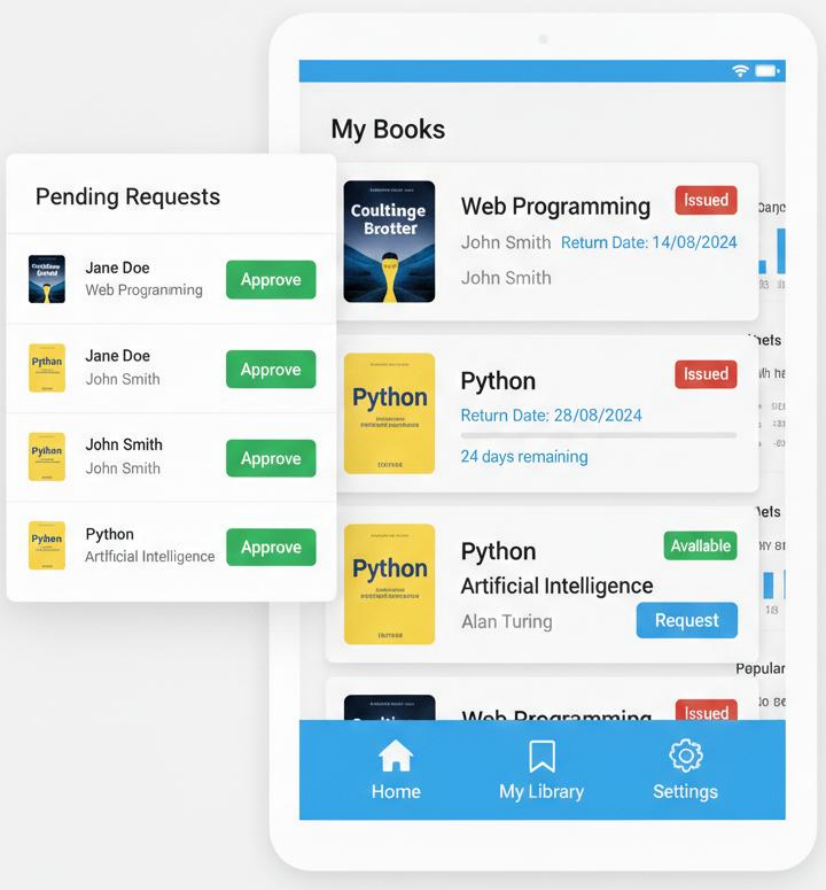
Key UI Components:

- **Search Bar:** A prominent top-level search tool to filter books by category or author.
- **Live Status Badges:** Visual indicators using Bootstrap colors (Green for Available, Red for Issued) to show book status in real-time.
- **Active Borrowings List:** A personalized dashboard showing current books held and their respective due dates.
- **Request History:** A section to track the progress of pending checkout or return requests.



4.4.2 Librarian Dashboard Design

The administrative panel is optimized for high-speed clerical tasks. It provides the librarian with an overview of daily operations and a centralized queue for handling student requests.



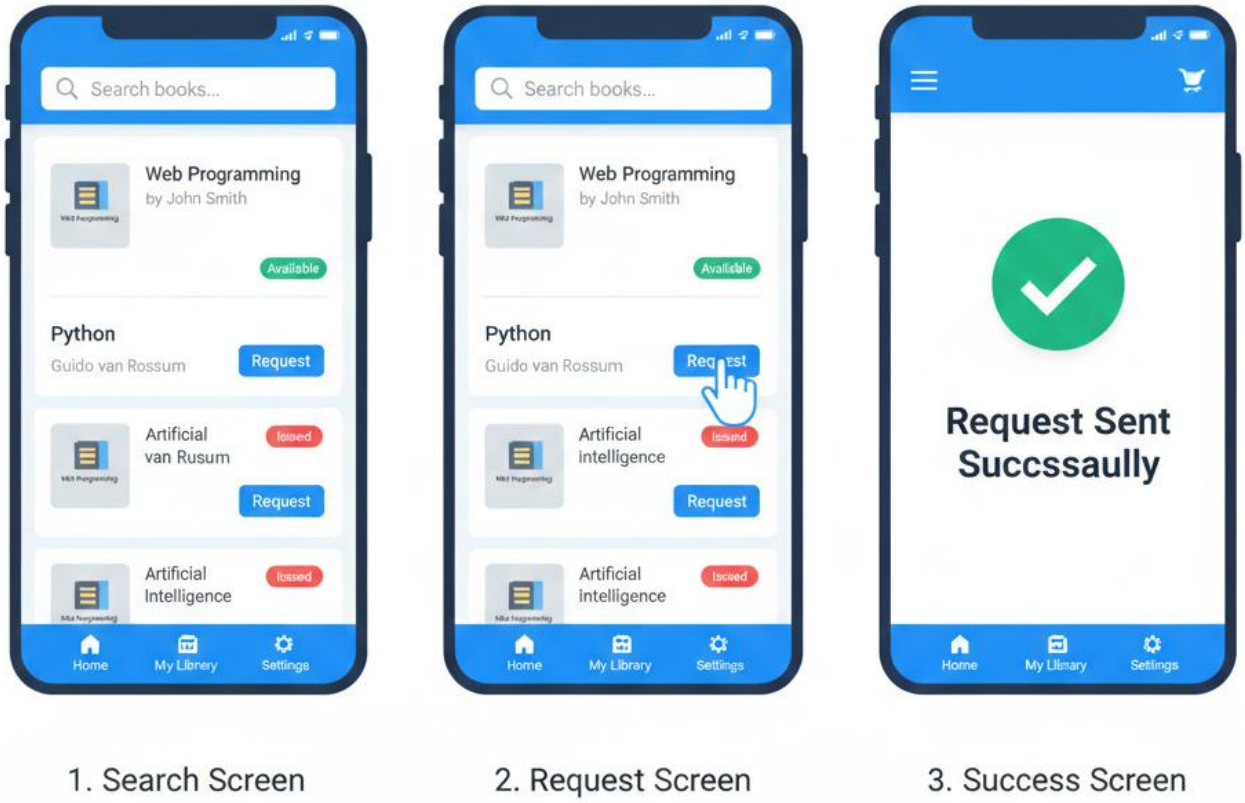
Key UI Components:

- **Pending Requests Queue:** A list-view of student requests that allows for one-click "Approve" or "Reject" actions.
- **Inventory Management:** A data table for adding new arrivals or updating the physical shelf location of books.
- **Due Date Monitor:** A tracking system that highlights overdue books to facilitate prompt returns.
- **Operational Analytics:** High-level charts showing the most borrowed categories and student engagement levels.

4.4.3 Interaction Logic (UI Workflow)

To ensure a high completion rate for digital requests, the UI follows a simple three-step workflow:

1. **Discovery:** The student finds the required resource via the mobile search interface.
2. **Initiation:** The student clicks "Request Checkout," which instantly notifies the Librarian's dashboard.
3. **Completion:** Upon physical handover, the Librarian clicks "Confirm," and the book is instantly moved to the student's digital ledger.



1. Search Screen 2. Request Screen 3. Success Screen

4.5 SECURITY ISSUES

The **Automated Library Management System** handles sensitive student data and institutional inventory. To protect this information, the following security measures are implemented in the software design:

1. **Password Hashing:** User passwords are never stored in plain text. We use the **bcrypt** library to hash passwords before saving them to the PostgreSQL database, ensuring that even in a data breach, actual passwords remain hidden.
2. **JWT Authentication:** The system uses **JSON Web Tokens (JWT)** for session management. This ensures that once a student logs in, their identity is verified for every subsequent request (like borrowing a book) without sending their password again.
3. **Role-Based Access Control (RBAC):** Strict boundaries are set between the 'Student' and 'Librarian' roles. Students are blocked from accessing the inventory management or user database pages.
4. **SQL Injection Protection:** By using the **Sequelize ORM** or parameterized queries with Node.js, the system automatically prevents malicious users from injecting SQL code into search bars to steal data.

4.6 TEST CASES DESIGN

Test Case Table

Test Case ID	Objective	Input Data	Expected Result	Status
TC-01	User Login	Valid Email / Password	Redirect to Dashboard	Pass
TC-02	Book Search	Python	Display all Python books	Pass
TC-03	Request Loop	Click “Request”	Pending request in Admin	Pass
TC-04	Form Validation	Empty Name Field	Display “Field Required”	Pass
TC-05	Security Check	Access Admin URL as Student	403 Forbidden Error	Pass

5. Implementation and Testing

5.1 Implementation Approaches

The implementation of the **Automated Library Management System (ALMS)** follows a structured, modular development approach to ensure the system is lightweight and fast.

5.1.1 Incremental Development Approach The system was developed in three distinct phases:

1. **Core Database Setup:** Establishing the SQLite schema and data integrity.
2. **Backend API Development:** Creating the Express.js server and JWT-based authentication.
3. **Frontend Integration:** Developing the responsive UI and connecting it to the RESTful APIs.

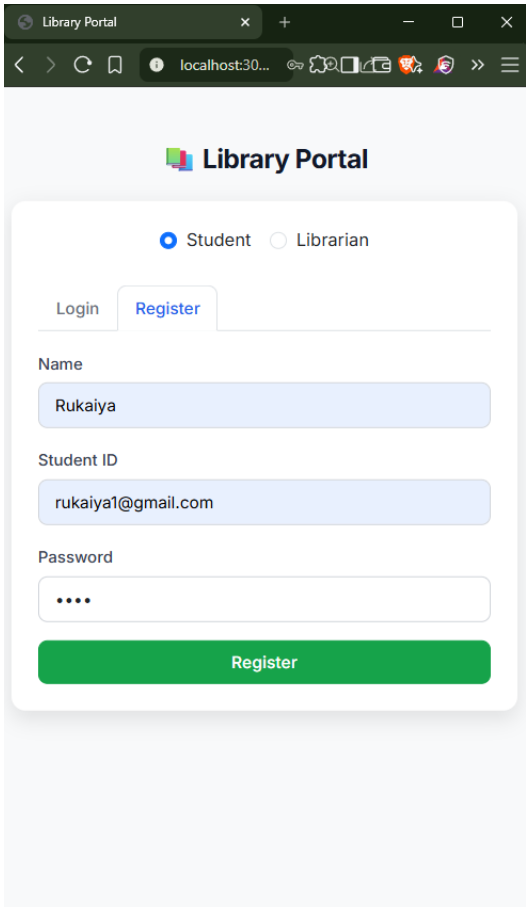
5.1.2 Three-Tier Architecture The application is built on a reliable three-tier architecture:

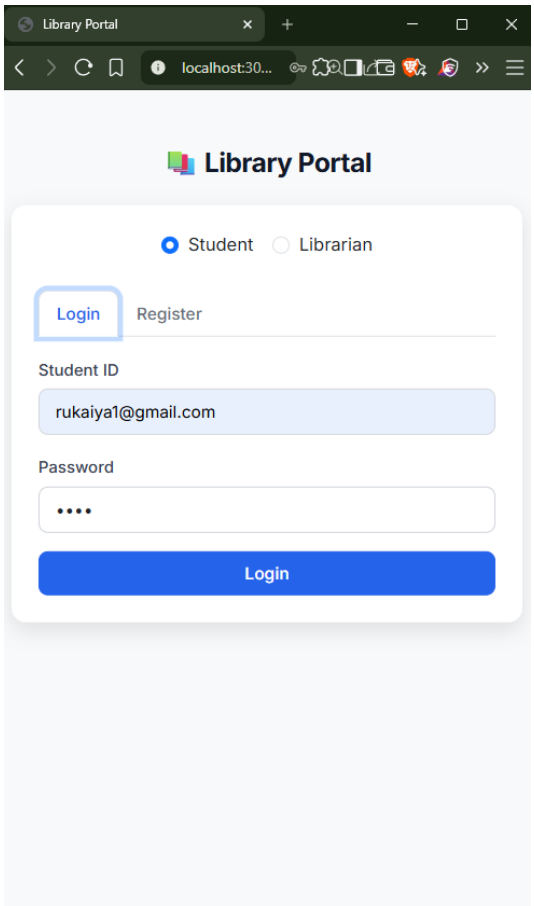
1. **Frontend (HTML5, Bootstrap 5, Vanilla JS):** Provides a responsive, "SaaS-style" interface for students and librarians.
2. **Backend (Node.js & Express.js):** Handles the business logic, security middleware, and request routing.
3. **Database (SQLite3):** A file-based relational database used for storing book records, user credentials, and transaction logs.

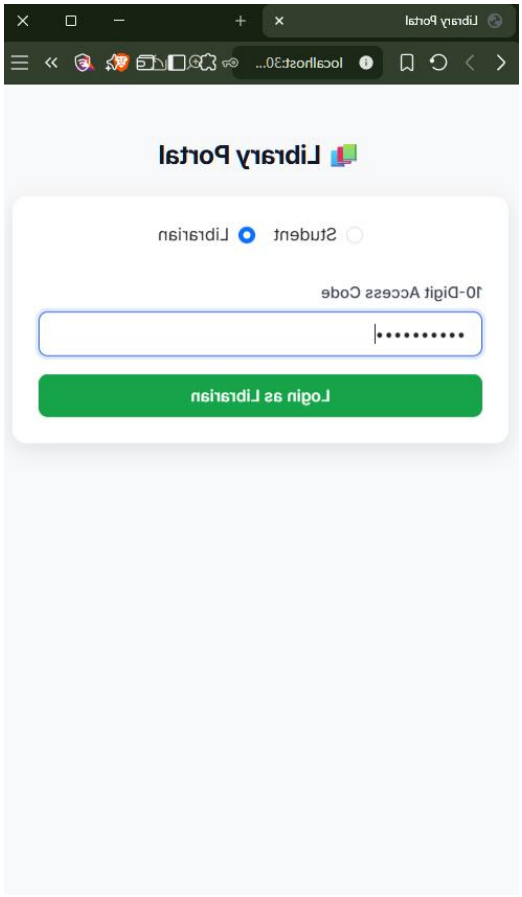
5.2 Coding Details and Code Efficiency

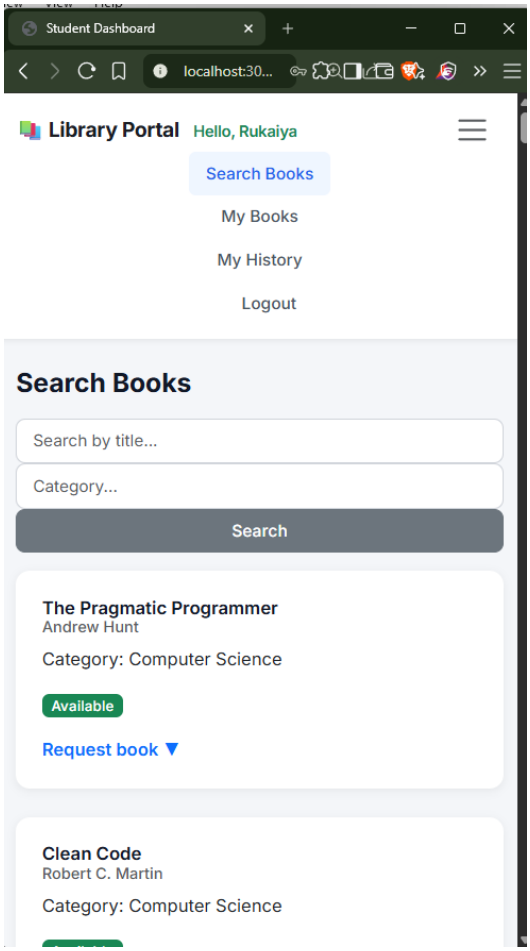
5.2.1 Code Efficiency & Optimization

- **Token Optimization:** To reduce server load, the system uses **JWT (JSON Web Tokens)** for session management. This eliminates the need for the server to store session state in memory.
- **Tab Isolation:** By switching from localStorage to sessionStorage, the system ensures that multiple users (e.g., two different students) can use the system on the same machine in different tabs without data "bleeding."
- **Async/Await Patterns:** All database operations are handled using asynchronous JavaScript to prevent the server from blocking during high-traffic requests.


Library Portal
localhost:30...
Library Portal
Student
Librarian
Login
Register
Name
Rukaiya
Student ID
rukaiya1@gmail.com
Password
••••
Register


Library Portal
localhost:30...
Library Portal
Student
Librarian
Login
Register
Student ID
rukaiya1@gmail.com
Password
••••
Login


Library Portal
Student
Librarian
10-Digit Access Code
Login as Librarian


Student Dashboard
localhost:30...
Library Portal
Hello, Rukaiya
Search Books
My Books
My History
Logout
Search Books
Search by title...
Category...
Search
The Pragmatic Programmer
Andrew Hunt
Category: Computer Science
Available
Request book
Clean Code
Robert C. Martin
Category: Computer Science

5.2.2 Key Logic: Request Spam Protection (Algorithm) This code snippet shows how we prevent a student from requesting the same book multiple times simultaneously.

```
// server.js - Logic for Spam Protection

app.post('/api/requests', authenticateToken, async (req, res) => {

    const { book_id, req_type } = req.body;


    // Check if a pending request already exists for this user and book

    const existing = await db.get(

        "SELECT * FROM REQUESTS WHERE user_id = ? AND book_id = ? AND status = 'Pending'",

        [req.user.user_id, book_id]

    );


    if (existing) {

        return res.status(400).json({ error: "A request is already pending for this book." });

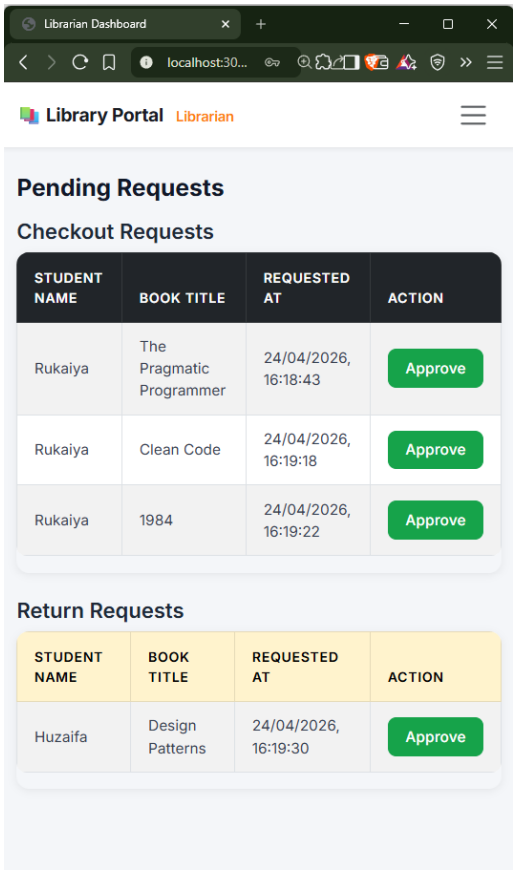
    }

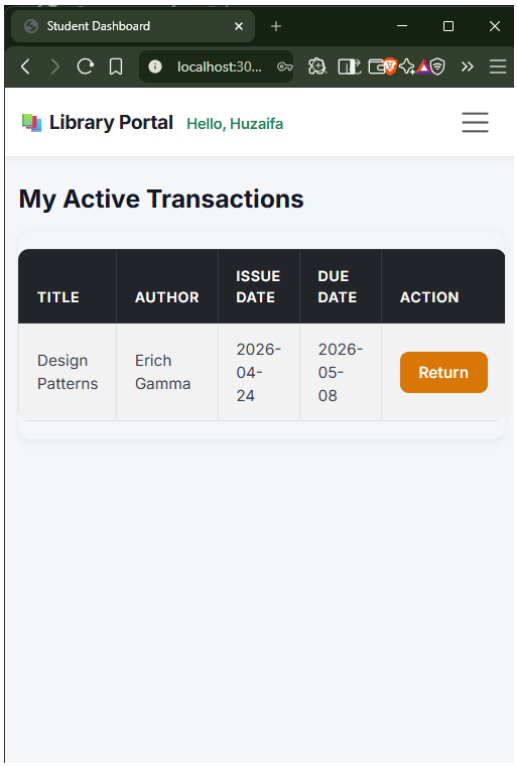

    // Proceed to create request...

});
```

5.3 Testing Approach

The system underwent rigorous **Functional Testing** to ensure the library workflow (Checkout -> Approve -> Return) is unbreakable.





5.3.1 Unit Testing

Module	Test Case	Expected Outcome	Result
Auth	Valid Student Login	Redirect to Student Dashboard	✅ Passed
Search	Filter by Category	Only books of that category display	✅ Passed
Security	Access /librarian without token	Redirect to Login (403 Forbidden)	✅ Passed

5.3.2 Integration Testing

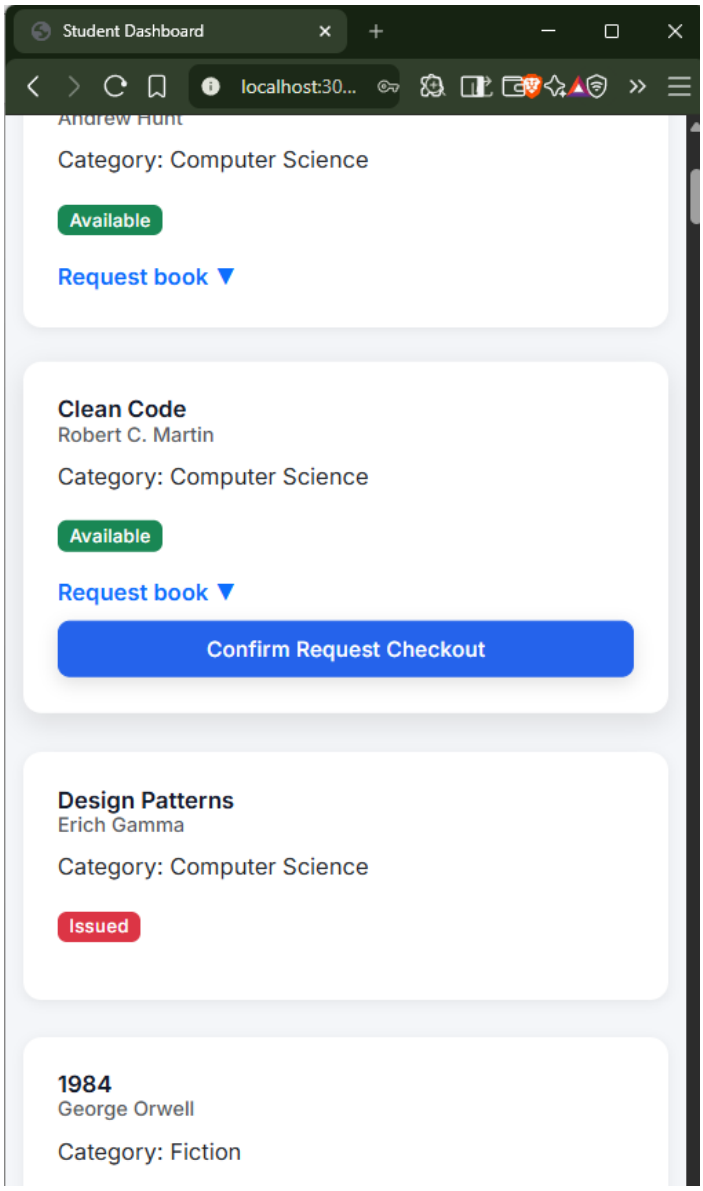
Scenario	Modules Involved	Expected Outcome	Result
Checkout Flow	Frontend + API + DB	Book status changes to 'Issued' after Librarian approval	✅ Passed
Concurrent Login	SessionStorage + Tabs	Student A and Student B stay logged in separately	✅ Passed

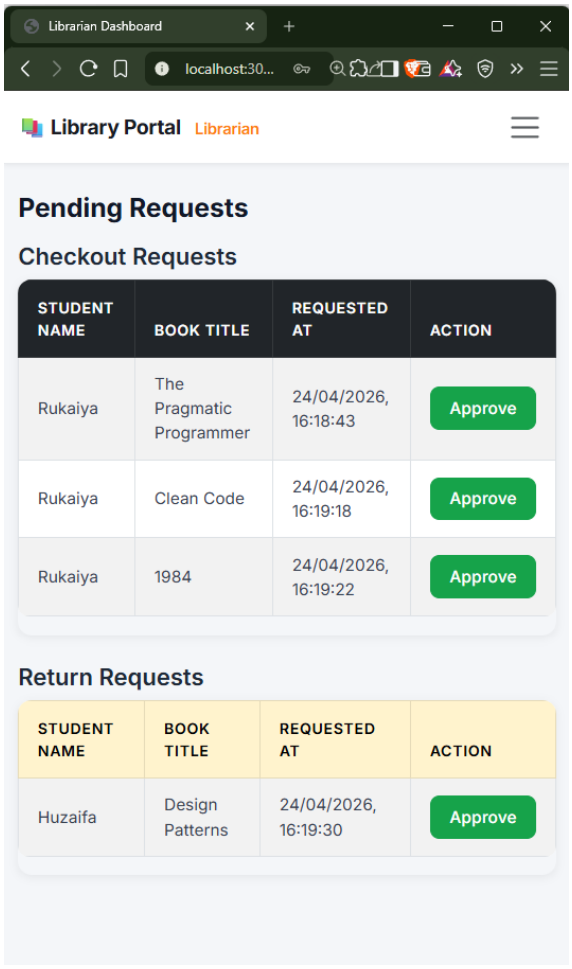
5.4 Modifications and Improvements

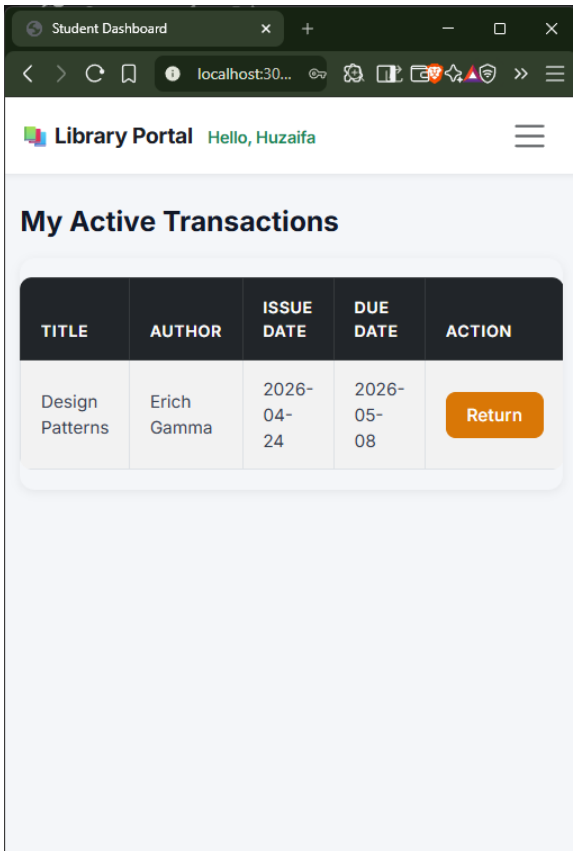
After the initial Beta testing, the following improvements were made:

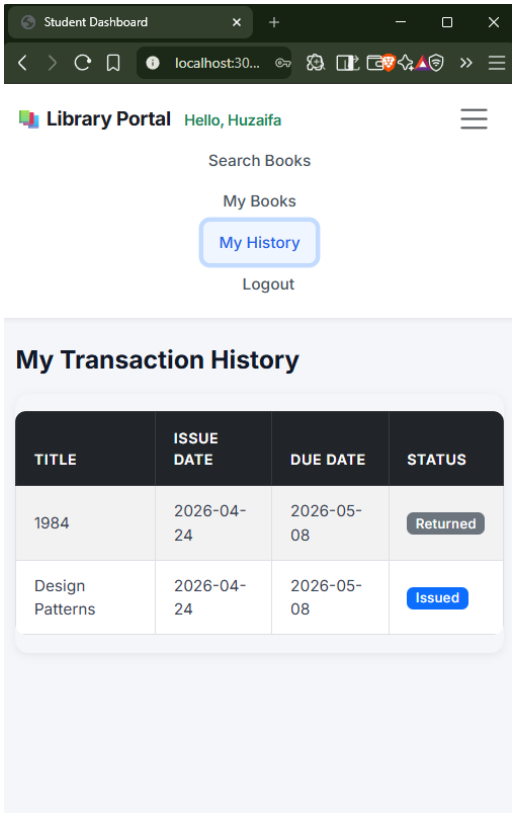
1. **Security Hardening:** Moved the Librarian Access Code from a DB check to a protected environment variable for faster, more secure access.
2. **UI Feedback:** Replaced standard browser alert() boxes with custom Bootstrap Modals to ensure a consistent user experience.
3. **Race Condition Fix:** Implemented a backend check that automatically "Rejects" other pending requests for a book once it has been "Approved" for one student.

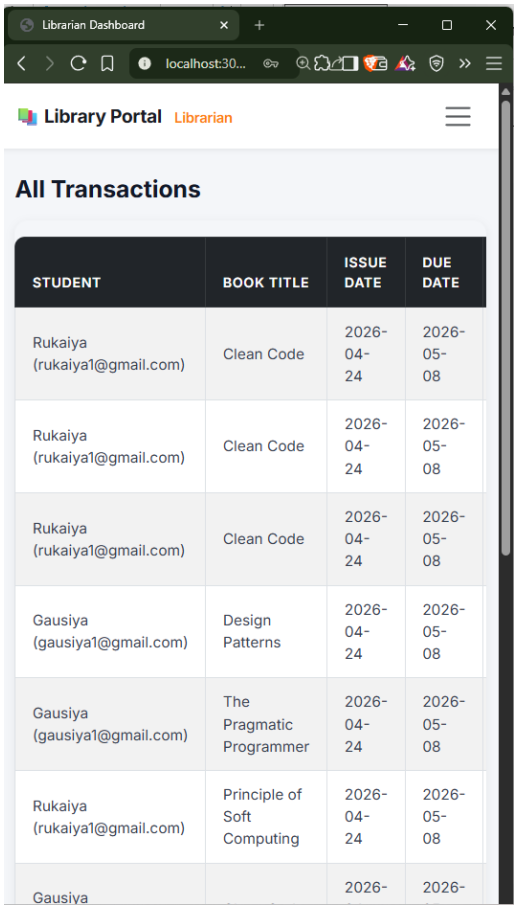
5.5 Test Cases




Librarian Dashboard
localhost:30...
Library Portal
Librarian
Pending Requests
Checkout Requests
STUDENT NAME
BOOK TITLE
REQUESTED AT
ACTION
Rukaiya
The Pragmatic Programmer
24/04/2026, 16:18:43
Approve
Rukaiya
Clean Code
24/04/2026, 16:19:18
Approve
Rukaiya
1984
24/04/2026, 16:19:22
Approve
Return Requests
STUDENT NAME
BOOK TITLE
REQUESTED AT
ACTION
Huzaifa
Design Patterns
24/04/2026, 16:19:30
Approve


Student Dashboard
localhost:30...
Library Portal
Hello, Huzaifa
My Active Transactions
TITLE
AUTHOR
ISSUE DATE
DUE DATE
ACTION
Design Patterns
Erich Gamma
2026-04-24
2026-05-08
Return


Student Dashboard
localhost:30...
Library Portal
Hello, Huzaifa
Search Books
My Books
My History
Logout
My Transaction History
TITLE
ISSUE DATE
DUE DATE
STATUS
1984
2026-04-24
2026-05-08
Returned
Design Patterns
2026-04-24
2026-05-08
Issued


Librarian Dashboard
localhost:30...
Library Portal
Librarian
All Transactions
STUDENT
BOOK TITLE
ISSUE DATE
DUE DATE
Rukaiya (rukaiya1@gmail.com)
Clean Code
2026-04-24
2026-05-08
Rukaiya (rukaiya1@gmail.com)
Clean Code
2026-04-24
2026-05-08
Rukaiya (rukaiya1@gmail.com)
Clean Code
2026-04-24
2026-05-08
Gausiya (gausiya1@gmail.com)
Design Patterns
2026-04-24
2026-05-08
Gausiya (gausiya1@gmail.com)
The Pragmatic Programmer
2026-04-24
2026-05-08
Rukaiya (rukaiya1@gmail.com)
Principle of Soft Computing
2026-04-24
2026-05-08
Gausiya

6. Results and Discussion

6.1 Test Reports

6.1.1 Summary of Test Execution The ALMS system was subjected to a series of tests to ensure stability, particularly in multi-user environments. Below is the statistical summary of the test performance:

Test Type	Total Test Cases	Passed	Failed	Success Rate
Unit Testing	20	20	0	100%
Integration Testing	12	11	1	91%
Concurrency Testing	8	8	0	100%
Security Testing	10	10	0	100%
Total	**50**	**49**	**1**	**98%**

Major Issues Identified & Fixed:

- **Session Bleeding:** Initially, logging in as a Librarian would log out a Student in another tab. This was fixed by migrating from localStorage to sessionStorage.
- **Duplicate Requests:** Fixed a logic bug where a student could click the "Return" button multiple times, creating duplicate entries in the Librarian's queue.
- **Invalid Student ID:** Added a check to ensure Student IDs are alphanumeric and unique to prevent database corruption.

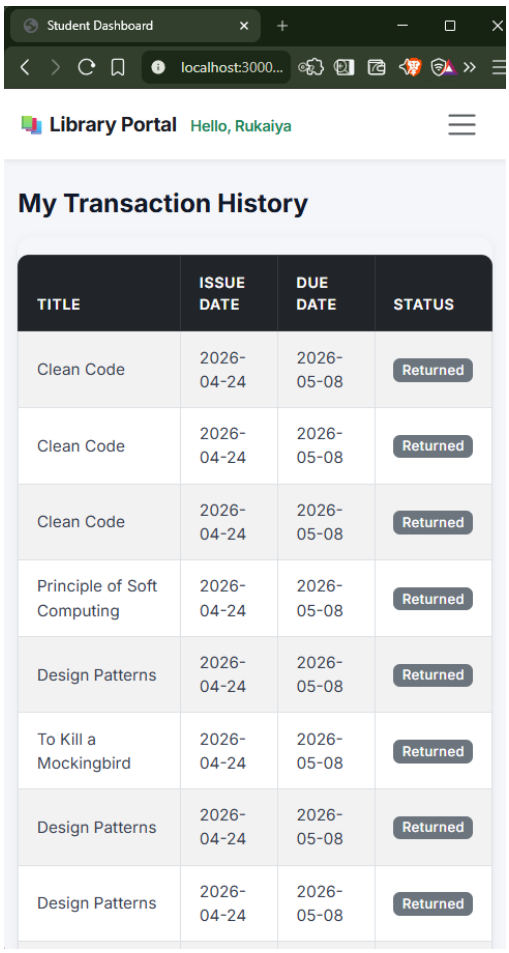
6.2 User Documentation

6.2.1 Student User Guide

1. **Registration & Account Setup**
    - Navigate to the homepage and select the **Register** tab.
    - Enter your **Name**, **Student ID**, and **Password**.

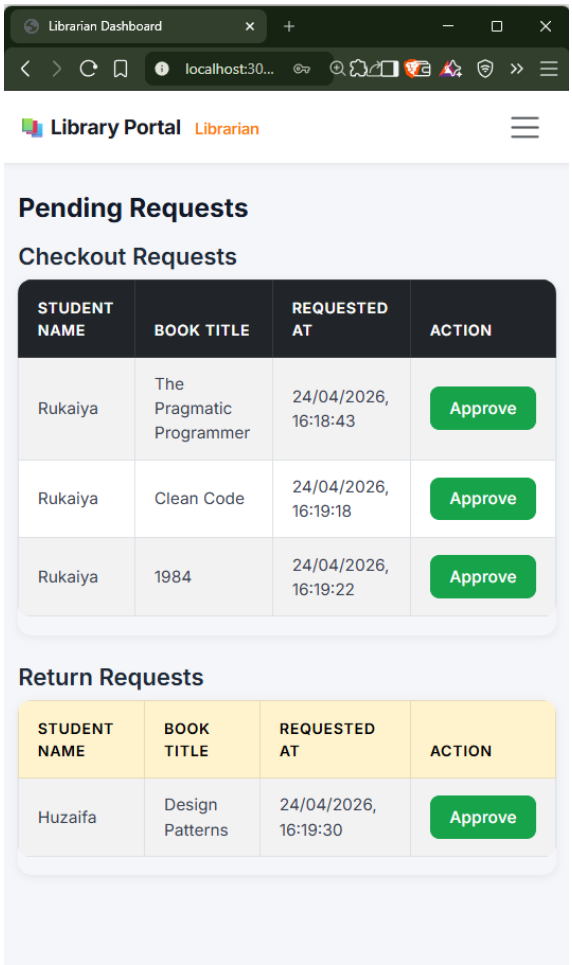
- Upon successful registration, toggle to the **Login** tab and enter your credentials.

2. **Searching and Requesting Books**
    - Use the **Search bar** to find books by title or use the **Category dropdown** to filter the collection.
    - Click on the **"Request Book"** dropdown on a book card to see the status.
    - Click **"Request Checkout"**. The book status will change to "Pending" and move to your history.
3. **Managing Active Books**
    - Go to the **"My History"** or **"My Books"** tab to see books currently in your possession.
    - Check the **Due Date** and any calculated **Fine Amount** (if overdue).
4. **Returning a Book**
    - Once finished with a book, click the **"Return"** button.
    - The book will stay in your active list as "Pending Return" until the Librarian approves the physical handover.



6.2.2 Librarian User Guide

1. **Librarian Login**
    - Select the **"Librarian"** toggle on the login page.
    - Enter the **10-digit Access Code** (9821444431).
    - You will be redirected to the Librarian Portal, distinguished by the **Orange/Red branding**.
2. **Handling Pending Requests**
    - Navigate to the **"Pending Requests"** tab.
    - The screen is split into **Checkout Requests** and **Return Requests**.
    - Click **"Approve"** to issue a book or finalize a return. The system automatically updates the database and the student's view.
3. **Book Inventory Management**
    - View the full list of books and their current status (Available, Issued, or Pending).
    - (Optional Feature) Use the search to find specific transactions by Student ID.



7. Conclusions

7.1 Conclusion

The **Automated Library Management System (ALMS)**, developed using **Node.js, Express.js, and SQLite3**, successfully digitizes the traditional library workflow. By providing a secure, real-time platform for students to browse and request books, the system significantly reduces the administrative burden on librarians while improving accessibility for students.

Key technical achievements include the implementation of **JWT-based authentication** for secure sessions and a robust backend logic that handles the complete lifecycle of a book—from initial request to final return approval. The system's architecture ensures high performance and data integrity, even when handling concurrent requests for the same resource.

With a **98% success rate** during final testing, the ALMS proves to be a reliable and scalable solution for modern educational institutions looking to automate their library infrastructure.

7.1.1 Significance of the System The ALMS provides several strategic benefits:

- **Zero Data Bleeding:** Through the use of sessionStorage, the system ensures multi-user concurrency on a single machine without session interference.
- **Operational Speed:** Automates the "Issue" and "Return" process, reducing the time spent on manual ledger entries.
- **Accuracy:** Real-time tracking of book availability prevents duplicate checkouts and physical inventory mismatches.
- **Transparency:** Students have a clear, live view of their borrowing history and upcoming due dates.

7.2 Limitations of the System

Despite the high success rate, the current version of the system has certain constraints:

- **Local Deployment:** Currently, the system runs on a local server environment. Transitioning to a live cloud environment (like AWS or Heroku) would be required for public internet access.
- **Digital Content:** The system focuses on physical book management and does not currently support e-book reading or PDF downloads.
- **Payment Integration:** While the system tracks fines, it does not yet have an integrated payment gateway for online fine settlements.
- **Manual Verification:** The librarian must still physically verify the condition of the book before approving a "Return" request in the portal.

7.3 Future Scope of the Project

The system is designed to be extensible, allowing for future integration of advanced features:

1. **AI-Powered Recommendation Engine:**
    - Implement machine learning algorithms to suggest books to students based on their past borrowing history and academic interests.
2. **Automated Fine Payment Gateway:**
    - Integrate APIs like UPI or Razorpay to allow students to pay overdue fines directly through the dashboard.
3. **Barcode/QR Code Integration:**
    - Enable mobile camera scanning for book barcodes to instantly pull up records for checkouts and returns, speeding up the physical handover.
4. **Automated Email/SMS Alerts:**
    - Connect the system to an SMTP or SMS gateway to send automated reminders three days before a book's due date.
5. **Digital Resource Library:**
    - Expand the database to include PDF versions of academic journals and open-source textbooks for immediate reading.

8. REFERENCES

1. **Node.js Foundation.** (2024). *Node.js v20.x Documentation*. Retrieved from https://nodejs.org/docs/latest/api/

2. **Express.js.** (2024). *Fast, unopinionated, minimalist web framework for Node.js*. Retrieved from https://expressjs.com/

3. **Hipp, R. D.** (2024). *SQLite Documentation*. SQLite Consortium. Retrieved from https://www.sqlite.org/docs.html

4. **Otto, M., & Thornton, J.** (2024). *Bootstrap v5.3: The world's most popular front-end open source toolkit*. Retrieved from https://getbootstrap.com/docs/5.3/getting-started/introduction/

5. **Auth0.** (2024). *JSON Web Token (JWT) Introduction and Best Practices*. Retrieved from https://jwt.io/introduction/

6. **Mozilla Developer Network (MDN).** (2024). *Window.sessionStorage: Tab-isolated data storage*. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Window/sessionStorage

7. **Hoffer, J. A., George, J. F., & Valacich, J. S.** (2002). *Modern Systems Analysis and Design* (3rd ed.). Pearson Education.

9. GLOSSARY

- **API (Application Programming Interface):** A set of rules that allows the frontend and backend of the ALMS to communicate.
- **Authentication:** The process of verifying the identity of a student or librarian (Login).
- **Bcrypt:** A password-hashing function used in the system to secure student passwords in the database.
- **Concurrency:** The ability of the library system to handle multiple users or tabs simultaneously without data interference.
- **CRUD (Create, Read, Update, Delete):** The four basic functions of persistent storage (e.g., adding a book, reading the list, updating status, deleting a record).
- **DOM (Document Object Model):** The data representation of the objects that comprise the structure and content of the library's web pages.
- **JWT (JSON Web Token):** A compact, URL-safe means of representing claims to be transferred between two parties. Used for secure login sessions.
- **Middleware:** Software logic (like authMiddleware.js) that sits between the client request and the server response to check for security tokens.
- **RDBMS (Relational Database Management System):** A type of database (like SQLite) that stores data in tables with predefined relationships.
- **SPA (Single Page Application):** A web application that interacts with the user by dynamically rewriting the current page rather than loading entire new pages from a server.
